華中科技大學

本科生毕业设计

基于多模态图检索增强的智能体工作流自主生成方法研究

院　　系　计算机科学与技术

专业班级　大数据 2201

姓　　名　詹熙至

学　　号　U202215549

指导教师　莫益军

2026 年 5 月 15 日

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包括任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

作者签名：　　2026 年 5 月 15 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保障、使用学位论文的规定，同意学校保留并向有关学位论文管理部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权省级优秀学士论文评选机构将本学位论文的全部或部分内容编入有关数据进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于 1、保　密 □，在　　年解密后适用本授权书

2、不保密 ☑。

（请在以上相应方框内打“✓”）

作者签名：　　2026 年 5 月 15 日

导师签名：　　2026 年 5 月 15 日

摘 要

随着大型语言模型（Large Language Model，LLM）能力的快速提升，以多智能体协作为核心的自动化工作流系统在复杂垂直领域任务中展现出广阔应用前景。然而，现有工作流自动生成方法面向电力系统等专业领域时仍存在两类关键问题：一方面，生成过程过度依赖 LLM 参数化知识，难以充分利用领域文档、设备关系和操作规程中的结构化知识；另一方面，子任务与智能体之间缺乏能力、工具和接口层面的量化匹配机制，容易造成角色分配不准确、工作流可解释性不足等问题。

针对上述问题，提出一种基于图检索增强生成（Graph Retrieval-Augmented Generation，GraphRAG）和三维量化匹配的智能体工作流自主生成方法。首先，设计面向电力领域的多模态知识图谱构建流程，通过实体关系抽取、视觉语言模型描述和 BGE-M3 向量嵌入，将电力领域文本、图像与流程图融合为统一图结构，并写入 Neo4j 图数据库。其次，设计基于子图扩展的 GraphRAG 检索方法，以语义向量检索获得种子节点，以广度优先遍历补充多跳邻域关系，形成结构化图谱上下文。进一步，构造子任务-智能体混合匹配算法，从能力嵌入相似度、工具覆盖率、输入输出兼容性三个维度建立加权评分函数，实现分解子任务到专用智能体的可解释映射。最后，设计工作流有向无环图（Directed Acyclic Graph，DAG）编排与三层迭代验证-修复机制，支持拓扑排序、循环检测、接口兼容性修复以及 LangGraph 格式导出。

围绕电网故障排查（OPS）、电力知识问答（KQA）和电力数据分析（DA）三类场景构建包含 30 条任务的分层基准测试集，并结合 PowerGridQA 问答数据和 OPSD 开放电力时序数据开展实验。实验结果表明：（1）在工作流可执行率上，所提方法和基线方法均达到 100%，验证了 DAG 验证机制的有效性；（2）三维量化匹配算法是角色分配准确率（RAA）的决定性因素，相比无匹配的随机分配基线在全场景均值上提升 36.9 个百分点（0.056 → 0.425），且在三类场景中均稳定成立；（3）经 BGE-M3 语义化改造的步骤完整性指标将覆盖率从词汇版的 0.18 量级提升至 0.77 量级，更准确地反映了中文电力工作流的步骤覆盖；（4）BFS 子图扩展在当前 4,337 节点的电力知识图谱上未带来稳定增益，反而引入噪

声节点扰动下游任务分解，提示后续应研究基于查询难度的自适应跳数控制方法。研究结果验证了基于图谱语义检索与三维量化匹配在专业领域工作流自动生成任务上的有效性。

关键词：多模态知识图谱；图检索增强生成；智能体工作流；子任务-智能体匹配；电力系统

Abstract

With the rapid advancement of Large Language Models (LLMs), automated workflow systems centered on multi-agent collaboration have demonstrated broad application prospects in complex domain-specific tasks. However, existing workflow generation methods face two key challenges in professional domains such as power systems. First, generation often relies excessively on the parametric knowledge of LLMs and fails to exploit structured knowledge from domain documents, equipment relations, and operating procedures. Second, the mapping from subtasks to agents lacks quantitative modeling of capabilities, tools, and interfaces, which may lead to inaccurate role assignment and limited workflow interpretability.

To address these challenges, this thesis proposes an autonomous agent workflow generation method that combines Graph Retrieval-Augmented Generation (GraphRAG) with three-dimensional quantitative subtask-agent matching. A multimodal knowledge graph construction pipeline is designed to integrate power-domain texts, images, and flowcharts into a unified graph structure through entity-relation extraction, vision-language descriptions, and BGE-M3 vector embeddings, with the graph stored in Neo4j. A subgraph-oriented GraphRAG retrieval method is then developed, where semantic vector search retrieves seed nodes and breadth-first traversal supplements multi-hop neighborhood relations to form structured graph context. Furthermore, a hybrid subtask-agent matching algorithm is constructed with a weighted scoring function over capability embedding similarity, tool coverage, and input-output compatibility, enabling explainable mapping from decomposed subtasks to specialized agents. Finally, a workflow Directed Acyclic Graph (DAG) orchestration and validation mechanism with iterative repair is designed to support topological sorting, cycle detection, interface conformance, and LangGraph export.

Experiments are conducted on a stratified benchmark of 30 tasks covering grid fault diagnosis (OPS), power knowledge question answering (KQA), and power data analysis (DA), using PowerGridQA and the OPSD open power time-series dataset. The

results show that: (1) all methods achieve 100% workflow executability, verifying the effectiveness of the three-layer DAG validation; (2) the three-dimensional matching algorithm is the decisive factor of role assignment accuracy (RAA), improving the average RAA by 36.9 percentage points over the no-matching baseline (0.056 → 0.425), with consistent gains across all three scenarios; (3) the BGE-M3 semantic upgrade of the step completeness metric raises measured coverage from the 0.18 level (lexical version) to the 0.77 level, more faithfully reflecting paraphrased Chinese power-domain steps; (4) breadth-first subgraph expansion does not yield stable gains on the current 4,337-node power knowledge graph and even introduces noise nodes that perturb downstream task decomposition, suggesting that adaptive hop control conditioned on query confidence is a promising direction. The findings verify the effectiveness of graph-based semantic retrieval combined with quantitative agent matching for workflow generation in professional domains.

Keywords: Multimodal Knowledge Graph, Graph Retrieval-Augmented Generation, Agent Workflow, Subtask-Agent Matching, Power Systems

目 录

摘 要 ..... **I**

Abstract ..... **III**

1 绪 论 **1**

1.1 课题背景、目的与意义 1

1.2 国内外研究现状 3

1.3 论文的主要内容与结构 5

1.4 本章小结 6

2 相关技术综述 **7**

2.1 知识图谱与图结构检索增强生成 7

2.2 多模态文档理解与知识抽取 8

2.3 多智能体系统与工作流编排 9

2.4 本章小结 10

3 多模态知识图谱构建与 GraphRAG 检索方法 **12**

3.1 多模态知识图谱构建方法 12

3.2 基于子图扩展的 GraphRAG 检索方法 14

3.3 本章小结 17

4 智能体匹配与工作流生成方法 **18**

4.1 子任务分解与智能体特征图 18

4.2 子任务-智能体混合匹配算法 19

4.3 工作流 DAG 编排与验证机制 22

4.4 系统整体架构 25

4.5 本章小结 26

5 实验与结果分析 **27**

5.1 实验环境与数据集 27

5.2 评估指标 30

5.3 实验结果与分析 30

5.4 本章小结 38

6 结论与展望 40

6.1 研究结论 40

6.2 研究局限性 41

6.3 未来研究方向 42

参考文献 43

致 谢 47

1 绪 论

本章首先介绍智能体工作流自动生成领域的研究背景与发展趋势，分析当前方法在专业领域知识利用和智能体匹配方面存在的核心挑战，随后综述国内外相关研究现状，最后阐明本研究的主要内容与论文结构。

1.1 课题背景、目的与意义

1.1.1 研究背景与趋势

近年来，以 GPT-4、Claude、DeepSeek 为代表的大型语言模型（Large Language Model，LLM）在自然语言理解、逻辑推理与代码生成等任务上取得了显著突破[1][2]，国内研究也开始系统总结大语言模型评测方法与应用风险[3]。在此基础上，研究者们将 LLM 的能力进一步延伸至智能体（Agent）系统，通过赋予模型工具调用、记忆管理与多步规划能力，构建能够自主完成复杂任务的智能体框架。

多智能体协作（Multi-Agent Collaboration）是当前智能体研究的重要方向。与单一智能体相比，多智能体系统通过角色分工与并行协作，能够处理更加复杂的长链路任务。LangGraph[4]、AutoGen[5] 等框架相继涌现，为多智能体工作流的编程式定义提供了基础支撑。然而，上述框架均依赖人工预先设计工作流拓扑，难以应对开放领域的动态任务需求。

与此同时，检索增强生成（Retrieval-Augmented Generation，RAG）技术通过将外部知识注入 LLM 的推理过程，有效缓解了模型幻觉问题，并显著提升了领域适配能力[6]。进一步地，以 Microsoft GraphRAG[7] 为代表的图结构检索方法，将知识组织为实体-关系图，利用图遍历实现多跳推理，在复杂问答和摘要任务上展现出超越传统向量检索的性能。

电力系统作为关系国计民生的重要基础设施，其运维过程涉及大量专业知识与复杂操作规程[8]。电力设备故障排查、保护定值校验、新能源消纳分析等任务，需要多个专业角色协同完成，天然契合多智能体工作流的应用场景。已有研究利用深度学习与知识图谱进行变电站设备故障诊断[9]，也有研究将知识图谱与

溯因推理结合用于智能电网故障诊断[10]。此外，电力系统存在以 CIM（Common Information Model）为代表的标准化知识本体，适合以知识图谱的形式进行结构化表达，为 GraphRAG 方法的应用提供了优质的数据基础。

1.1.2 面临的问题与挑战

尽管 LLM 在通用任务上表现出色，将其直接用于电力等专业领域的工作流自动生成时，仍面临以下核心挑战：

（1）领域知识获取不足。LLM 的参数化知识主要来源于互联网通用语料，对专业领域内的最新标准规程、设备配置文档和故障案例覆盖有限。在缺乏外部知识注入的情况下，生成的工作流往往缺乏专业性和可操作性。

（2）多模态知识融合困难。电力系统文档普遍包含文本、表格、单线图和流程图等多种信息形态。现有的 RAG 方法大多仅处理纯文本，难以有效利用图像和图形所蕴含的结构化信息，导致知识抽取不完整。

（3）智能体匹配缺乏系统化机制。现有多智能体框架通常依赖 LLM 自身的理解能力进行隐式的角色分配，缺乏对智能体能力（Capability）、工具（Tool）和接口规范（Schema）的显式建模与量化匹配，造成角色分配的随机性与低准确率。

（4）工作流质量难以保证。自动生成的工作流可能存在循环依赖、数据流断裂等结构性错误，以及逻辑顺序不合理等语义性问题，缺乏系统性的验证机制。

1.1.3 课题目的与意义

针对上述挑战，本研究旨在构建一套完整的多模态 GraphRAG 驱动的智能体工作流自主生成方法，实现对专业领域任务的准确分解、知识增强和角色分配。具体目标如下：

一是构建面向电力领域的多模态知识图谱，支持文本、图像与流程图的统一表示和向量检索；二是设计基于子图扩展的 GraphRAG 检索方法，并定量分析向量种子检索与多跳邻域扩展在下游任务中的独立贡献；三是提出量化的子任务-智能体匹配算法，从能力、工具、接口三个维度对智能体进行评分与优选；

四是实现工作流的 DAG 编排与迭代验证-修复机制，保证生成工作流的可执行性和逻辑合理性。

本研究的成果可为电力调度、故障应急处置、知识服务等场景提供可靠的智能体工作流自动化支撑，具有重要的工程应用价值；同时，所提方法具有良好的领域通用性，可推广至医疗、法律、金融等其他专业领域。

1.2 国内外研究现状

1.2.1 检索增强生成方法

检索增强生成（RAG）由 Lewis 等人[6] 于 2020 年正式提出，通过在推理阶段动态检索外部文档并将其作为上下文输入 LLM，显著缓解了模型的事实幻觉问题。早期 RAG 方法基于稠密向量检索（Dense Passage Retrieval，DPR），以 FAISS 等近似最近邻索引为核心，在开放域问答任务上取得了良好效果。

随后，研究者从检索粒度、融合策略和迭代机制等多个维度对 RAG 进行了拓展。Self-RAG[11] 引入反思机制，使模型能够动态判断是否需要检索；CRAG[12] 提出上下文自适应压缩策略，提升了长文档场景下的检索效率；Adaptive RAG[13] 根据问题复杂度自适应选择检索策略。上述方法均以向量相似度为核心度量，在多跳推理和结构化知识利用方面存在固有局限。

1.2.2 基于图结构的知识检索

知识图谱（Knowledge Graph，KG）为结构化知识的存储和推理提供了天然载体，国内学者已对知识图谱的知识抽取、表示、融合和推理技术进行系统梳理[14]，并对知识表示学习的发展进行了总结[15]。KGRAG[16] 将传统 RAG 与知识图谱结合，通过实体链接将检索问题转化为子图查询，在复杂关系推理场景下显著优于向量检索。GraphRAG[7] 由微软研究院提出，在构建图的基础上引入社区摘要（Community Summary），实现了从局部到全局的层次化知识表示，在长文档摘要任务上展现出卓越性能。

RAPTOR[17] 通过递归文本摘要树构建多层次上下文索引，G-RAG[18] 引入图神经网络对检索节点进行语义重排。近期，HippoRAG[19] 借鉴人类海马体记忆机

制，构建个性化知识图谱以支持长期记忆检索。国内关于知识图谱与大语言模型融合的综述也指出，结构化知识有助于缓解幻觉并提升可解释性[20]。然而，上述方法多针对纯文本信息，缺乏对图像、流程图等多模态信息的处理能力，难以直接应用于电力系统等多模态文档丰富的领域。

1.2.3 多智能体工作流框架

多智能体系统的研究最早可追溯至分布式人工智能领域，近年随 LLM 的突破而获得新的发展动力。AutoGen[5] 设计了可对话的智能体编程接口，支持多智能体协作完成代码生成、数据分析等任务。LangGraph 基于有向图的计算模型，将智能体工作流抽象为节点（Node）和边（Edge），支持条件分支和循环结构的精确控制。MetaGPT[21] 将软件工程中的角色分工引入多智能体系统，通过标准化输入输出协议规范智能体间通信。

TaskWeaver[22] 提出以代码片段为执行单元的智能体编排机制，适合数据分析类任务；CAMEL[23] 探索了基于角色扮演的多智能体对话范式；CrewAI[24] 以”船员”隐喻组织智能体团队，支持任务的层级分解和并行执行。然而，上述框架均需人工预定义工作流拓扑，缺乏根据任务描述和领域知识自动生成最优工作流的能力。

1.2.4 专业领域智能体应用

将 LLM 和多智能体系统应用于电力等专业领域是近期研究的重要趋势。Cheng 等[25] 提出面向电力调度的 Grid Artificial Intelligent Assistant（GAIA），验证了大语言模型在运行调整、运行监视和黑启动等任务中的应用潜力。在知识图谱方面，变电站设备故障诊断[9] 和智能电网故障诊断[10] 等研究表明，结构化知识表示能够提升故障信息组织和推理分析能力。

然而，现有专业领域智能体方法大多孤立解决知识获取或任务执行的单一问题，尚未形成从多模态知识图谱构建到工作流自动生成的完整闭环方法。本研究正是针对这一空白提出系统性解决方案。

1.3 论文的主要内容与结构

1.3.1 论文的主要内容

本研究围绕”多模态 GraphRAG 驱动的智能体工作流自主生成”这一核心命题，开展以下四方面工作：

（1）多模态知识图谱构建（算法 A）。设计面向电力领域的多模态文档解析流水线，支持文本、图像和流程图三种模态的实体与关系抽取；构建统一的图节点表示，利用 BGE-M3 嵌入模型生成高维语义向量，并写入 Neo4j 图数据库建立向量索引。

（2）GraphRAG 检索方法（算法 B）。提出以语义相似度检索为种子、BFS 子图扩展为骨干的两阶段检索策略；引入 LLM 重排序机制对候选节点按相关性过滤，并设计结构化上下文格式化方法，将子图信息有效编码为 LLM 可理解的文本上下文。本研究在 30 条电力基准任务上系统评估了不同检索阶段对下游工作流生成的实际影响，量化了向量种子检索与 BFS 子图扩展的独立贡献边界。

（3）子任务-智能体匹配算法（算法 C）。基于智能体特征图（Agent Feature Graph），定义能力嵌入相似度、工具覆盖率和输入输出兼容性三维评分函数；提出工具覆盖率软约束机制和消融实验权重配置，实现对最优智能体的系统化选择。

（4）工作流编排与验证（算法 D）。基于 LLM 推断隐式数据流依赖，构建工作流有向无环图；设计三层迭代验证-修复机制（DAG 无环性、IO 兼容性、任务完整性，最大迭代次数 5），并实现 Mermaid 图、BPMN 2.0 和 LangGraph 三种格式的工作流导出。

1.3.2 论文结构

本文的组织结构如下：

第一章为绪论，介绍研究背景、问题挑战、国内外现状及论文结构。

第二章为相关工作，系统综述多模态 RAG、知识图谱构建和多智能体框架等领域的核心技术进展，分析与本研究的关联与区别。

第三章为方法论，详细阐述多模态知识图谱构建（算法 A）和 GraphRAG 检索（算法 B）的技术设计，包括节点表示、向量索引和子图扩展等关键步骤。

第四章为智能体匹配与工作流生成，重点介绍子任务-智能体匹配算法（算法 C）和工作流 DAG 编排与验证机制（算法 D），并说明系统整体架构。

第五章为实验与分析，基于电力系统三类场景（OPS/KQA/DA）评估所提方法的有效性，与四种基线方法进行对比，并开展消融实验。

第六章为结论与展望，总结本研究的主要贡献，分析现有方法的局限性，并展望未来研究方向。

1.4 本章小结

本章介绍了多模态 GraphRAG 驱动的智能体工作流自动生成研究的背景与意义，分析了当前领域在知识融合、智能体匹配和工作流质量保证三个层面存在的核心挑战，综述了检索增强生成、图结构知识检索、多智能体框架和专业领域应用四个方向的国内外研究现状，并阐明了本文的主要研究内容与章节结构。

2 相关技术综述

本章系统梳理与本研究直接相关的四个技术领域：知识图谱与图结构检索增强生成、多模态文档理解与知识抽取、多智能体系统与工作流编排，以及专业领域智能体应用，分析各方向的技术演进脉络及其与本研究的关联，为后续方法设计奠定理论基础。

2.1 知识图谱与图结构检索增强生成

2.1.1 传统 RAG 方法及其局限性

检索增强生成（RAG）由 Lewis 等人[6] 于 2020 年提出，通过在推理阶段动态检索外部文档并将其作为上下文输入 LLM，显著降低了模型幻觉问题。早期 RAG 系统以稠密向量检索（Dense Passage Retrieval，DPR）为核心，利用 FAISS 等近似最近邻算法在向量空间中检索语义相近段落。

然而，向量检索 RAG 存在固有局限：其一，检索粒度以独立文本块（chunk）为单位，缺乏跨文本的关系推理能力；其二，相似度度量仅捕捉浅层语义，难以处理多跳推理问题；其三，孤立段落的上下文丢失导致检索结果碎片化，影响生成质量。Self-RAG[11] 引入自反思机制以动态决定是否检索，CRAG[12] 提出上下文自适应压缩策略，Adaptive RAG[13] 根据问题复杂度自适应切换检索策略，上述工作均在向量检索框架内进行改进，未能从根本上解决多跳推理能力的不足。

2.1.2 基于知识图谱的结构化检索

知识图谱（Knowledge Graph，KG）为结构化知识存储和关系推理提供了天然载体。知识图谱技术综述[14] 和知识表示学习研究[15] 表明，实体、关系和语义表示的统一建模是知识推理与智能问答的重要基础。知识图谱与 RAG 的结合（KGRAG）[16] 通过实体链接将检索转化为子图查询，在关系推理场景下显著优于向量检索。KGRAG 方法首先识别问题中的实体，随后在图中定位对应节点，通过图遍历抽取相关子图作为上下文，实现了从孤立文档到结构化知识的检索范式转换。

微软研究院提出的 GraphRAG[7] 进一步引入社区检测和社区摘要（Community Summary）机制，通过 Leiden 算法划分图社区，并为每个社区生成摘要，实现从局部到全局的层次化知识表示。GraphRAG 在长文档摘要任务上展现出超越传统向量检索的性能，尤其适合需要全局理解的问答场景。

HippoRAG[19] 借鉴人类海马体记忆机制，构建个性化知识图谱支持长期记忆检索；RAPTOR[17] 通过递归文本摘要树构建多层次上下文索引。上述方法均展示了图结构在知识组织和多跳检索方面的优势。

2.1.3 现有图 RAG 方法的不足

尽管图结构 RAG 在文本知识检索上取得了重要进展，现有方法仍存在以下不足：第一，检索策略多为全局社区摘要或固定跳数遍历，缺乏以语义向量检索为种子的自适应子图扩展机制；第二，上述方法均针对纯文本信息，无法处理图像、流程图等多模态信息；第三，检索结果的格式化方式缺乏面向下游 LLM 推理的结构化设计。本研究提出的 GraphRAG 方法针对上述不足进行了系统性改进，将在第三章详述。

2.2 多模态文档理解与知识抽取

2.2.1 视觉语言模型与多模态理解

随着视觉语言模型（Vision-Language Model，VLM）的快速发展，文档图像的自动理解成为可能。CLIP[26] 通过对比学习建立图文语义对齐，为多模态表示学习奠定了基础。GPT-4V[1]、LLaVA[27] 等多模态大模型进一步实现了对复杂图表、流程图和表格的理解与描述能力。

在文档解析方面，DocLayoutYOLO[28] 和 TableTransformer[29] 等专用模型分别针对文档版式检测和表格结构解析进行了优化。这些工作共同推动了多模态文档理解从简单 OCR 向深度语义理解的演进。

2.2.2 多模态实体抽取与图谱构建

传统知识图谱构建依赖于对纯文本的命名实体识别(NER)和关系抽取(RE),对图像和图形中的结构化信息无能为力。近年来，研究者开始探索多模态实体抽取方法。MMC[30] 通过大规模图表指令数据提升模型对图表的跨模态理解能力；在电力故障诊断场景中，已有研究将多模态知识图谱与视觉检测模型结合，用于变电站设备故障的自主预警和诊断[9]。

在电力系统领域，单线图（Single Line Diagram，SLD）和网络拓扑图是重要的知识载体，蕴含丰富的设备连接关系。然而，现有方法尚未能有效解析电力系统专有图纸，将其中的实体（变压器、线路、断路器等）和关系（连接、保护等）自动转化为图谱节点和边。本研究利用 VLM 对电力图像进行描述，再通过 LLM 进行实体-关系抽取，实现了电力多模态文档的统一知识图谱构建。

2.2.3 嵌入模型与多语言语义表示

知识图谱的向量化检索依赖高质量的文本嵌入模型。BGE-M3[31] 由智源研究院提出，支持超过 100 种语言的语义嵌入，在单向量检索、稀疏检索和多向量交互三种检索模式上均取得了优异性能。BGE-M3 的多粒度表示能力使其在中英文混合的电力领域文档中具有显著优势，本研究选用 BGE-M3 生成 1024 维语义嵌入，并通过 Neo4j 向量索引实现高效的近似最近邻检索。

2.3 多智能体系统与工作流编排

2.3.1 基于 LLM 的多智能体框架

多智能体系统（Multi-Agent System，MAS）的研究最早源于分布式人工智能领域，近年随 LLM 能力的突破而获得新的发展动力。AutoGen[5] 设计了以可对话智能体为核心的编程接口，通过智能体间的多轮对话协作完成代码生成、数据分析等复杂任务，并支持人机混合协作模式。

MetaGPT[21] 将软件工程中的角色分工引入多智能体系统，通过标准化输入输出协议规范智能体间通信，定义了产品经理、架构师、工程师等专业角色，实

现了端到端的软件开发自动化。CAMEL[23] 探索了基于角色扮演的多智能体对话范式，通过提示工程引导智能体扮演特定角色。CrewAI[24] 以”船员”隐喻组织智能体团队，支持任务的层级分解和并行执行。

2.3.2 工作流编排框架

LangGraph[4] 基于有向图（Directed Graph）的计算模型，将智能体工作流抽象为节点（Node）和边（Edge），支持条件分支、循环结构和并行子图的精确控制，并提供状态持久化和检查点机制。TaskWeaver[22] 以代码片段为执行单元的智能体编排机制，通过将用户请求转化为 Python 代码任务提升执行可靠性。

上述框架均依赖人工预先设计工作流拓扑，需要开发者以编程方式指定智能体间的调用关系。这种方式灵活性高，但难以应对开放域的动态任务需求——不同用户请求可能需要完全不同的工作流结构，人工预设无法覆盖所有场景。

2.3.3 工作流自动生成方法

工作流自动生成旨在根据任务描述，无需人工干预地生成最优智能体协作方案。现有研究主要依赖基于 LLM 的直接生成，通过提示工程或思维链机制引导 LLM 输出工作流结构[32]，但生成质量依赖模型内部知识，缺乏对外部领域知识和智能体能力约束的显式利用。

在智能体匹配方面，现有框架大多依赖 LLM 隐式的角色分配能力，缺乏对智能体能力（Capability）和工具（Tool）的显式建模与量化匹配。本研究提出的子任务-智能体匹配算法（算法 C），通过三维评分函数系统化解决角色分配准确率问题，将在第四章详细介绍。

2.4 本章小结

本章从检索增强生成、多模态文档理解和多智能体框架三个维度梳理了相关技术的发展现状。在图结构 RAG 方面，GraphRAG 等方法在文本知识检索上取得了重要进展，但缺乏多模态处理能力和面向工作流生成的子图自适应扩展机制；在多模态理解方面，VLM 和 BGE-M3 为多模态知识图谱构建提供了技术

基础；在多智能体编排方面，LangGraph 等框架提供了图计算支撑，但自动工作流生成和系统化智能体匹配仍是待解难题。本研究正是在上述基础上，提出涵盖知识图谱构建、GraphRAG 检索、智能体匹配和工作流编排四个环节的完整方法体系，各核心算法将在后续章节详细阐述。

3 多模态知识图谱构建与 GraphRAG 检索方法

本章详细阐述本研究的前两项核心方法：面向电力领域的多模态知识图谱构建方法（算法 A）和基于子图扩展的 GraphRAG 检索方法（算法 B）。两者共同构成智能体工作流生成系统的知识底座，为后续的任务分解与智能体匹配提供结构化的领域上下文。

3.1 多模态知识图谱构建方法

3.1.1 整体框架设计

面向电力领域的知识图谱构建面临多模态异构文档的处理挑战。电力系统文档通常混合包含技术规程文本、设备参数表格、单线图和操作流程图，传统纯文本方法只能处理其中一种模态，造成知识抽取严重不完整。

本研究设计的多模态文档解析流水线如图3-1所示，整体分为三个阶段：（1）多模态解析阶段，对输入文档按模态分流处理；（2）实体-关系抽取阶段，利用 LLM 从各模态信息中抽取实体和关系三元组；（3）图谱写入阶段，将抽取结果统一表示为图节点和边，写入 Neo4j 图数据库并建立向量索引。

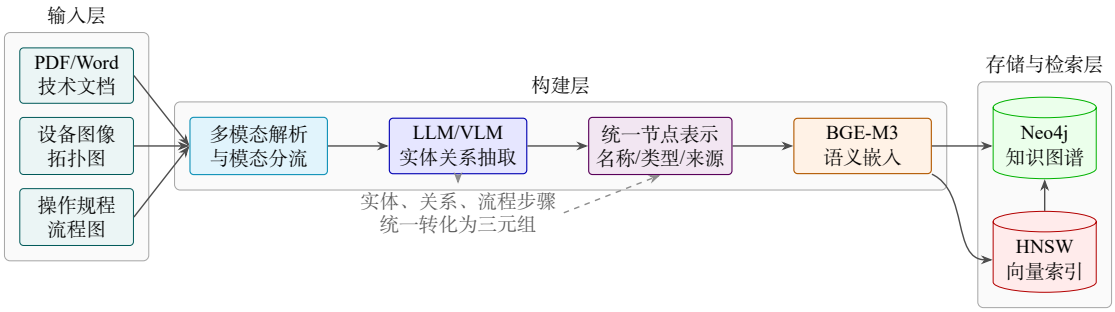

图 3-1 多模态知识图谱构建流程

3.1.2 多模态文档解析

文本模态处理。对于 PDF 和 Word 格式的文本文档，采用滑动窗口分块策略将长文本切分为语义完整的段落，每块长度设定为 512 tokens，块间重叠 64

tokens 以保证上下文连续性。随后调用 LLM 以结构化提示抽取实体（设备名称、技术参数、标准编号等）及实体间关系（”所属”、”连接”、”保护”等），输出为三元组列表。

图像模态处理。对于电力设备照片和拓扑图，首先利用视觉语言模型（GPT-4V）生成图像的自然语言描述，详细描述图像中可见的设备、连接关系和标注信息。随后将生成的描述文本与原图配对，输入 LLM 进行实体-关系抽取，确保图像信息以结构化形式进入知识图谱。

流程图模态处理。对于操作规程中的流程图，采用类似图像的处理路径，借助 VLM 识别流程节点（操作步骤、判断条件、执行结果）和有向边（执行顺序、条件分支），将流程图转化为有序的操作步骤序列，并抽取步骤间的顺序关系和条件关系。

3.1.3 统一节点表示与向量索引

所有模态的抽取结果统一映射为图节点表示。每个图节点包含以下属性：节点标识符（id）、节点类型（type，取值为 Entity/Relation/Process/Concept）、节点名称（name）、节点描述（description）、节点来源（source，记录原始文档路径和页码）以及语义嵌入向量（embedding）。

语义嵌入向量由 BGE-M3 模型[31] 生成，维度为 1024。BGE-M3 支持中英文双语语义编码，适合电力领域中文技术文档的处理需求。嵌入生成时，将节点名称与描述拼接为”[name] [description]” 格式后送入模型，保证嵌入向量同时捕捉节点的简短标识和详细语义。

节点和关系写入 Neo4j 图数据库后，利用 Neo4j 5.x 内置的向量索引（Vector Index）功能，在 embedding 属性上建立 HNSW（Hierarchical Navigable Small World）向量索引，支持高效的近似最近邻检索。索引参数配置为：相似度度量余弦（cosine），HNSW 最大连接数 m=16，ef_construction=64。

最终构建的电力领域知识图谱包含 4,337 个节点和 5,228 条关系，覆盖变压器、断路器、保护装置等核心电力设备，以及故障类型、操作规程、技术标准等领域概念。

3.1.4 算法 A 的流程描述

算法 A（多模态知识图谱构建）的完整流程如算法3-1所示。

算法 3-1 多模态知识图谱构建算法（算法 A）

Input: 多模态文档集合 $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$，LLM $\mathcal{M}$，VLM $\mathcal{V}$，嵌入模型 $\mathcal{E}$
Output: 知识图谱 $G = (V, E)$，向量索引 Index

```
初始化空节点集 V ← ∅，空边集 E ← ∅
for all 文档 d_i ∈ D do
    按模态分类文档内容 → {文本块，图像，流程图}
    for all 文本块 t do
        T ← M(extract_prompt(t))
        将 T 中的实体和关系加入 V, E
    end for
    for all 图像 img do
        desc ← V(img)
        T ← M(extract_prompt(desc))
        将 T 中的实体和关系加入 V, E
    end for
end for
for all 节点 v ∈ V do
    v.embedding ← E(v.name || v.description)
end for
将 (V, E) 写入 Neo4j 图数据库
在 {v.embedding | v ∈ V} 上建立 HNSW 向量索引 Index
return G = (V, E)，Index
```

3.2 基于子图扩展的 GraphRAG 检索方法

3.2.1 检索方法总体设计

传统 RAG 以文本块为检索单元，丢失了实体间的结构化关系信息。本研究提出的 GraphRAG 检索方法以知识图谱为底层数据结构，采用”语义向量检索种子节点 + BFS 子图扩展”的两阶段策略，实现多跳关系推理和完整上下文召回。

检索流程分为四个步骤：（1）查询向量化；（2）语义相似度种子检索；（3）

BFS 子图扩展；（4）LLM 重排序与上下文格式化。图3-2展示了以种子节点为中心的子图扩展与上下文形成过程。

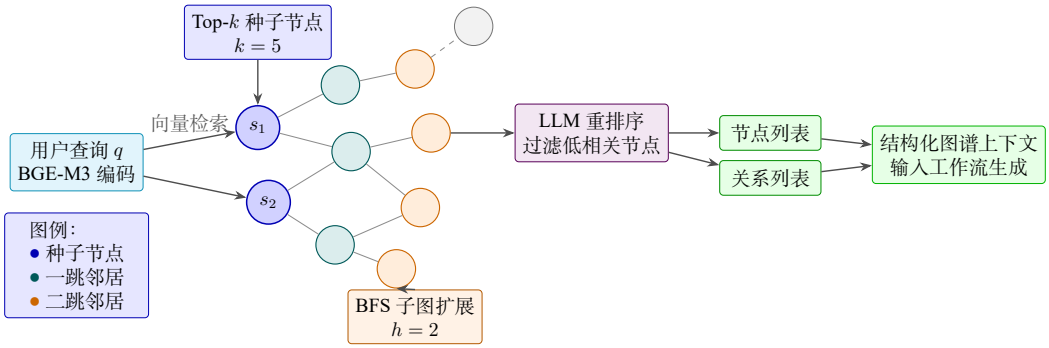

图 3-2　GraphRAG 种子检索与子图扩展示意

3.2.2　查询向量化与种子节点检索

给定用户查询 q，首先利用与索引建立时相同的 BGE-M3 模型将其编码为查询向量 $\mathbf{q} = \mathcal{E}(q) \in \mathbb{R}^{1024}$。随后在 Neo4j 向量索引上执行近似最近邻检索，获取与查询语义最相近的 k 个种子节点：

$$S = \text{TopK}_{v \in V}\left(\text{cosine}(\mathbf{q}, v.\text{embedding}), k\right) \tag{3-1}$$

其中 k 为超参数（实验中设 $k = 5$），余弦相似度定义为：

$$\text{cosine}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \tag{3-2}$$

种子节点集 S 代表与查询最直接相关的图节点，作为后续子图扩展的起点。

3.2.3　BFS 子图扩展

仅依赖种子节点的局部信息不足以支持多跳推理，因此需要沿图结构扩展上下文范围。本研究采用广度优先搜索（BFS）从种子节点出发，遍历 h 跳邻域（实验中设 $h = 2$）：

$$\mathcal{G}_{\text{sub}} = \text{BFS}(S, G, h) = \bigcup_{v \in S} \{u \in V \mid d_G(v, u) \leq h\} \tag{3-3}$$

其中 $d_G(v,u)$ 表示图 G 中节点 v 到 u 的最短路径长度。子图 $\mathcal{G}_{\text{sub}}$ 包含所有在 h 跳范围内可达的节点及其连接边，完整保留了局部图结构信息。

BFS 扩展的时间复杂度为 $\mathcal{O}(k \cdot (d^h - 1)/(d-1))$，其中 d 为图的平均度数。对于电力知识图谱，实验测量平均度数 $d \approx 2.4$，$k = 5, h = 2$ 时扩展节点数约为 35 个，检索时延在可接受范围内。

3.2.4 LLM 重排序与上下文格式化

BFS 扩展可能引入与查询关联度较低的噪声节点。为此，引入 LLM 重排序机制：将查询 q 与每个候选节点的名称和描述拼接后输入 LLM，由 LLM 输出 0 到 1 之间的相关度分数，过滤掉分数低于阈值 θ（实验设 $\theta = 0.3$）的节点，保留相关度最高的 m（实验设 $m = 10$）个节点。

最终，将保留节点及其间的关系格式化为结构化文本上下文，格式设计如下：

```
【知识图谱上下文】
节点 1: [类型] 名称: 描述
节点 2: [类型] 名称: 描述
...
关系: 节点A --[关系类型]--> 节点B
关系: 节点C --[关系类型]--> 节点D
...
```

上述格式通过类型标注和关系显式表达，使 LLM 能够清晰理解图谱结构，有效利用关系信息进行推理。

3.2.5 算法 B 的流程描述

算法 B（GraphRAG 检索）的完整流程如算法3-2所示。

算法 3-2 GraphRAG 子图扩展检索算法（算法 B）

Input: 查询 q，知识图谱 $G = (V, E)$，向量索引 Index，参数 k, h, θ, m

Output: 结构化上下文 ctx

1: 查询向量化：$\mathbf{q} \leftarrow \mathcal{E}(q)$
2: 种子节点检索：$S \leftarrow \text{TopK}(\text{cosine}(\mathbf{q}, \cdot), k)$ via Index
3: BFS 子图扩展：$\mathcal{G}_{\text{sub}} \leftarrow \text{BFS}(S, G, h)$
4: LLM 重排序：对 $\mathcal{G}_{\text{sub}}$ 中每个节点计算相关度分数
5: 过滤低相关节点（阈值 θ），保留 Top-m 节点
6: 提取子图中节点间关系边
7: 格式化上下文：将节点列表和关系列表转为结构化文本 ctx
8: **return** ctx

3.3 本章小结

本章详细阐述了多模态知识图谱构建（算法 A）和 GraphRAG 检索（算法 B）两项核心方法。算法 A 通过对文本、图像和流程图三种模态的统一解析，利用 BGE-M3 嵌入模型和 Neo4j 向量索引，构建了包含 4,337 个节点的电力领域知识图谱；算法 B 以语义向量检索为种子节点，结合 BFS 子图扩展和 LLM 重排序，实现了多跳关系推理和精准上下文召回。两项方法共同解决了领域知识利用不足和多模态信息融合困难的核心挑战，为后续的子任务分解与智能体匹配提供了高质量的领域知识支撑。第四章将在此基础上，介绍子任务-智能体匹配算法和工作流编排与验证方法。

4 智能体匹配与工作流生成方法

本章介绍本研究的后两项核心方法：子任务-智能体混合匹配算法（算法 C）和工作流 DAG 编排与验证机制（算法 D），并阐述系统整体架构。算法 C 解决了”将分解后的子任务准确映射到最优专用智能体”的核心难题，算法 D 负责将匹配结果编排为可执行的有向无环图工作流并保证其正确性。

4.1 子任务分解与智能体特征图

4.1.1 任务分解方法

给定用户任务描述 T 和 GraphRAG 检索到的领域上下文 ctx，首先通过 LLM 将任务分解为若干子任务 $\{st_1, st_2, \ldots, st_n\}$，每个子任务 st_i 包含以下属性：子任务标识符（id）、描述（description）、所需能力列表（required_capabilities）、所需工具列表（required_tools）、输入类型（input_type）、输出类型（output_type）以及依赖关系（depends_on）。

分解提示词（Decomposition Prompt）设计为：将原始任务、领域上下文和子任务 JSON Schema 作为输入，要求 LLM 输出结构化的子任务列表，并明确各子任务间的数据依赖关系。依赖关系以 depends_on 字段记录前驱子任务的 id 列表，为后续 DAG 构建提供基础。

4.1.2 智能体特征图定义

智能体特征图（Agent Feature Graph，AFG）是本研究对多智能体系统中智能体能力的形式化表示。AFG 以异构有向图 $\mathcal{F} = (\mathcal{A} \cup \mathcal{V}_C \cup \mathcal{V}_T,\ \mathcal{E}_{cap} \cup \mathcal{E}_{tool})$ 表示，其中：

- 智能体节点集 $\mathcal{A} = \{a_1, a_2, \ldots, a_m\}$ 表示可用专用智能体；
- 能力节点集 $\mathcal{V}_C = \bigcup_{a_i \in \mathcal{A}} \text{capabilities}(a_i)$ 由所有智能体声明能力的并集构成；
- 工具节点集 $\mathcal{V}_T = \bigcup_{a_i \in \mathcal{A}} \text{tools}(a_i)$ 由所有智能体可用工具的并集构成；
- 有向边 $\mathcal{E}_{cap} = \{(a_i, c) \mid c \in \text{capabilities}(a_i)\}$ 表示 *HAS_CAPABILITY* 关系，连接智能体到其声明能力；

- 有向边 $\mathcal{E}_{tool} = \{(a_i, t) \mid t \in \text{tools}(a_i)\}$ 表示 *USES_TOOL* 关系，连接智能体到其可用工具。

该异构结构与算法 A 构建的领域知识图谱共享 Neo4j 数据库存储（具体节点标签 `AgentNode/CapabilityNode/ToolNode` 与边类型 `HAS_CAPABILITY/USES_TOOL`），便于在算法 C 的能力嵌入相似度计算中直接复用领域图谱已建立的 BGE-M3 向量索引。

每个智能体 a_i 具有以下属性：智能体标识符（id）、描述（description）、能力列表（capabilities）、工具列表（tools）、输入 Schema（input_schema）和输出 Schema（output_schema）。能力和工具均以自然语言描述，并通过 BGE-M3 嵌入为语义向量，存储为 $\mathbf{e}_{a_i} \in \mathbb{R}^{1024}$。

在电力系统场景中，本研究定义了七类专用智能体：故障诊断智能体（FaultDiagnosisAgent）、保护定值校验智能体（ProtectionAgent）、数据分析智能体（DataAnalysisAgent）、知识检索智能体（KnowledgeRetrievalAgent）、文档生成智能体（DocGenerationAgent）、可视化智能体（VisualizationAgent）以及通用协调智能体（OrchestratorAgent），共同覆盖电力运维的主要任务场景。

4.2 子任务-智能体混合匹配算法

4.2.1 三维评分函数设计

针对每对（子任务 st_i，智能体 a_j），算法 C 从三个维度计算匹配分数，加权求和得到综合评分 $\text{Score}(st_i, a_j)$：

$$\text{Score}(st_i, a_j) = \alpha \cdot s_{\text{cap}} + \beta \cdot s_{\text{tool}} + \gamma \cdot s_{\text{io}} \tag{4-1}$$

其中 α, β, γ 为权重超参数，满足 $\alpha + \beta + \gamma = 1$；$s_{\text{cap}}$、$s_{\text{tool}}$、$s_{\text{io}}$ 分别为能力嵌入相似度、工具覆盖率和输入输出兼容性三个分量。

（1）能力嵌入相似度 s_{cap}。将子任务描述和所需能力列表拼接为能力查询向量 $\mathbf{q}_{st_i}$，与智能体能力描述嵌入向量 $\mathbf{e}_{a_j}$ 计算余弦相似度：

$$s_{\text{cap}}(st_i, a_j) = \text{cosine}\left(\mathcal{E}(st_i.\text{description} + st_i.\text{required_caps}),\ \mathbf{e}_{a_j}\right) \tag{4-2}$$

能力嵌入相似度从语义层面衡量子任务需求与智能体能力的匹配程度，捕捉表述不同但语义相近的能力描述之间的对应关系。

（2）工具覆盖率 s_{tool}。工具覆盖率衡量智能体所拥有的工具是否满足子任务的工具需求：

$$s_{\text{tool}}(st_i, a_j) = \frac{|\text{Required_Tools}(st_i) \cap \text{Tools}(a_j)|}{|\text{Required_Tools}(st_i)| + \epsilon} \tag{4-3}$$

其中 ϵ 为防除零的平滑项。当子任务不需要特定工具时（Required_Tools 为空），s_{tool} 取默认值 1.0。此外，设定工具覆盖率软约束阈值 τ_{tool}，当 $s_{\text{tool}} < \tau_{\text{tool}}$ 时将该智能体标记为次优候选，在最终排序中降权处理（实验中 $\tau_{\text{tool}} = 0.0$ 以保证候选多样性）。

（3）输入输出兼容性 s_{io}。s_{io} 通过对子任务声明的输入/输出类型与智能体 Schema 字段名进行双向子串匹配来度量接口契合度。给定一个数据类型描述 d 和 Schema S，定义单边匹配函数：

$$\mu(d, S) = \begin{cases} 1.0 & \exists k \in \text{keys}(S) : \ d \sqsubseteq k \ \vee \ k \sqsubseteq d \\ 0.5 & \text{否则} \end{cases} \tag{4-4}$$

其中 $\sqsubseteq$ 表示不区分大小写的子串包含关系。s_{io} 由输入侧与输出侧的单边匹配取均值得到：

$$s_{\text{io}}(st_i, a_j) = \tfrac{1}{2}\big[\mu(st_i.\text{input_type},\ a_j.\text{input_schema}) + \mu(st_i.\text{output_type},\ a_j.\text{output_schema})\big] \tag{4-5}$$

设计上，未在 Schema 中找到对应字段时给予 0.5 的中等分而非 0 分，避免接口描述空缺导致候选智能体被过早淘汰；当字段存在双向子串匹配时给予满分，体现”声明的类型与目标 Schema 至少存在词汇层重叠”的兼容性证据。该方案是面向工程实现的轻量替代——基于 LLM 的三级语义评分版本能够进一步识别”JSON 结构化”与”自然语言文本”等跨表示冲突，但会在每对（子任务，智能体）匹配中引入约 0.5 秒额外延迟，导致算法 C 的端到端开销随智能体规模线性放大，因此本研究保留 LLM 版本作为未来工作（详见 6.2 节局限性 (5)）。

4.2.2 最优智能体选择

对每个子任务 st_i，遍历所有非通用智能体候选 $a_j \in \mathcal{A}$，计算三维综合评分，选择得分最高的智能体作为最优匹配：

$$a_i^* = \max_{a_j \in \mathcal{A}} \text{Score}(st_i, a_j) \tag{4-6}$$

若所有候选智能体的最高综合评分均低于绝对阈值 τ_{abs}（实验设为 0.2），则分配通用协调智能体（OrchestratorAgent）作为回退选项，保证每个子任务均有智能体承接。

最终，算法 C 为每个子任务输出三元组 $(st_i, a_i^*, \text{Score}_i)$，记录子任务、最优智能体和匹配分数，传递给算法 D 进行工作流编排。

4.2.3 算法 C 的流程描述

算法 C 的输入包括原始任务 T、GraphRAG 检索得到的领域上下文 ctx、智能体特征图 $\mathcal{F}$ 以及权重参数 (α, β, γ)，输出为子任务到智能体的匹配结果集合 $\mathcal{R}$。其核心思想是将 LLM 的开放式任务分解与可计算的三维匹配函数结合起来：首先利用 ctx 约束 LLM 生成结构化子任务集合，降低分解结果脱离电力领域知识的风险；随后对每个子任务遍历候选智能体，分别计算能力嵌入相似度、工具覆盖率和输入输出兼容性；最后按照加权综合评分选择最优智能体，并在所有候选均低于阈值时回退到通用协调智能体。

因此，算法 C 既保留了 LLM 对复杂任务的语义拆解能力，又通过显式评分函数避免了角色分配完全依赖 LLM 自由生成的问题。图4-1给出了算法 C 的流程化表示，算法4-1给出了对应伪代码。

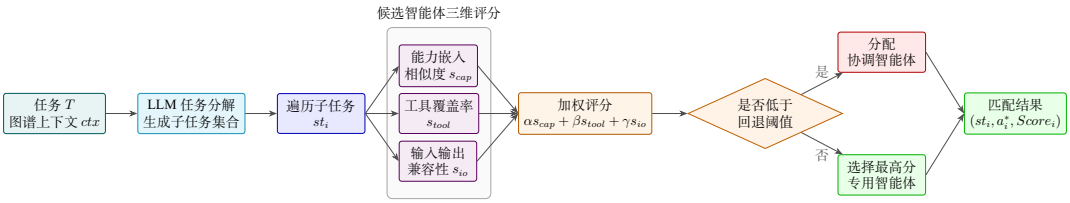

图 4-1 子任务-智能体混合匹配流程图

算法 4-1 子任务-智能体混合匹配算法（算法 C）

Input: 任务 T，领域上下文 ctx，智能体特征图 $\mathcal{F}$，权重 (α, β, γ)

Output: 匹配结果列表 $\{(st_i, a_i^*, \text{Score}_i)\}$

子任务分解：$\{st_1, \ldots, st_n\} \leftarrow \text{LLM_Decompose}(T, ctx)$
初始化结果列表 $\mathcal{R} \leftarrow [\,]$
for all 子任务 st_i **do**
    初始化 best_score $\leftarrow 0$，best_agent $\leftarrow$ None
    for all 智能体 $a_j \in \mathcal{A}$，$a_j \neq$ OrchestratorAgent **do**
        $s_{\text{cap}} \leftarrow \text{cosine}(\mathcal{E}(st_i), \mathbf{e}_{a_j})$
        $s_{\text{tool}} \leftarrow |\text{Req}(st_i) \cap \text{Tools}(a_j)| \,/\, (|\text{Req}(st_i)| + \epsilon)$
        $s_{\text{io}} \leftarrow \text{IOMatch}(st_i, a_j)$
        score $\leftarrow \alpha \cdot s_{\text{cap}} + \beta \cdot s_{\text{tool}} + \gamma \cdot s_{\text{io}}$
        if score $>$ best_score **then**
            best_score $\leftarrow$ score；best_agent $\leftarrow a_j$
        end if
    end for
    if best_score $< \tau_{\text{abs}}$ **then**
        best_agent $\leftarrow$ OrchestratorAgent
    end if
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{(st_i, \text{best_agent}, \text{best_score})\}$
end for
return $\mathcal{R}$

4.3 工作流 DAG 编排与验证机制

4.3.1 DAG 构建方法

基于算法 C 的匹配结果 $\{(st_i, a_i^*, \text{Score}_i)\}$，算法 D 构建工作流有向无环图（Directed Acyclic Graph，DAG）。DAG 中每个节点对应一个工作流步骤（WorkflowStep），节点属性包括步骤标识符（step_id）、智能体标识符（agent_id）、执行动作描述（action）、输入参数（inputs）以及依赖前驱步骤列表（depends_on）。

依赖关系的确定依赖两个来源：一是子任务分解时的显式数据流依赖（depends_on 字段）；二是 LLM 对隐式数据流的推断。对于显式依赖未能覆盖的情况，利用 LLM 分析每对步骤间是否存在数据传递关系（即步骤 i 的输出是否

为步骤 j 的输入)，若存在则在 DAG 中添加有向边 $i \to j$。

4.3.2 算法 D 的流程描述

算法 D 的输入为算法 C 输出的匹配结果集合 $\mathcal{R} = \{(st_i, a_i^*, \text{Score}_i)\}$、原始任务 T 以及子任务分解阶段得到的显式依赖集合 $\mathcal{D}_{exp}$，输出为经过验证的工作流 DAG 及其可部署导出结果。算法 D 首先把每个匹配三元组转换为一个工作流节点，并保留子任务描述、执行智能体、输入输出约束和匹配置信度等属性；然后结合显式依赖与 LLM 推断得到的隐式数据流依赖构造有向边集合，形成初始 DAG。

在初始图生成后，算法 D 进入“验证–修复”循环：若图结构存在环路，则删除或重排导致环路的依赖边；若相邻节点的输入输出 Schema 不兼容，则插入类型转换步骤或调整智能体分配；若工作流未完整覆盖原始任务目标，则追加缺失步骤并重新连接依赖关系。只有当结构、接口和语义三类约束全部满足后，算法 D 才导出 Mermaid 图、BPMN 2.0 定义和 LangGraph 代码。图4-2给出了算法 D 的流程化表示，算法4-2给出了对应伪代码。

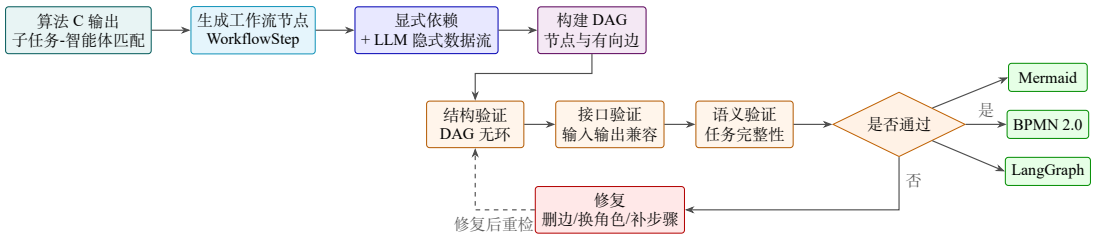

图 4-2 工作流 DAG 编排与三层验证流程图

算法 4-2 工作流 DAG 编排与三层验证算法（算法 D）

Input: 匹配结果 $\mathcal{R}$，原始任务 T，显式依赖集合 $\mathcal{D}_{exp}$

Output: 验证后的工作流 DAG $G = (V, E)$，导出结果 $\mathcal{O}$

```
V ← BuildWorkflowSteps(R)
E ← D_exp
for all 工作流节点对 (v_i, v_j) do
    if LLM_DataFlow(v_i, v_j) = 1 then
        E ← E ∪ {(v_i, v_j)}
    end if
end for
G ← (V, E), valid ← false
while valid = false do
    if HasCycle(G) then
        G ← RepairCycle(G)
    else if ¬IOCompatible(G) then
        G ← RepairInterface(G)
    else if ¬TaskComplete(G, T) then
        G ← RepairCoverage(G, T)
    else
        valid ← true
    end if
end while
O ← Export(G, {Mermaid, BPMN, LangGraph})
return G, O
```

4.3.3 三层验证机制

为保证生成工作流的正确性和可执行性，设计三层渐进式验证机制：

第一层：DAG 无环性验证（结构验证）。利用拓扑排序算法（Kahn 算法）检测工作流图中是否存在循环依赖。若检测到环路（如步骤 A 依赖步骤 B，步骤 B 又依赖步骤 A），则定位并删除引入环路的边，直至工作流成为合法 DAG。此层验证保证工作流可被线性调度执行，是可执行性的基础保障。

第二层：输入输出兼容性验证（接口验证）。对 DAG 中每条有向边 $i \to j$，验证步骤 i（由智能体 a_i^* 执行）的输出 Schema 是否与步骤 j（由智能体 a_j^* 执行）

的输入 Schema 兼容。若存在类型不匹配，通过插入类型转换步骤（Type Adapter）或调整智能体分配来修复接口冲突。

第三层：任务完整性验证（语义验证）。对比原始任务描述 T 与工作流步骤集合，由 LLM 判断工作流是否完整覆盖任务的所有关键目标。若存在任务目标遗漏，则追加对应工作流步骤，确保生成工作流的语义完整性。

4.3.4 工作流格式导出

完成验证后，算法 D 支持将工作流导出为三种标准格式：

（1）**Mermaid 图**：以 flowchart TD 语法生成人类可读的流程图定义，可在 Markdown 文档中直接渲染；

（2）**BPMN 2.0**：以 XML 格式导出符合业务流程模型与标记法（Business Process Model and Notation）标准的工作流定义，支持主流 BPMN 工具（如 Camunda、Activiti）解析执行；

（3）**LangGraph 代码**：生成基于 LangGraph 框架的 Python 代码，可直接部署执行，每个工作流步骤对应一个 LangGraph 节点，依赖关系对应图边。

4.4 系统整体架构

图4-3展示了多模态 GraphRAG 驱动的智能体工作流自主生成系统的整体架构。系统分为离线构建阶段和在线生成阶段。

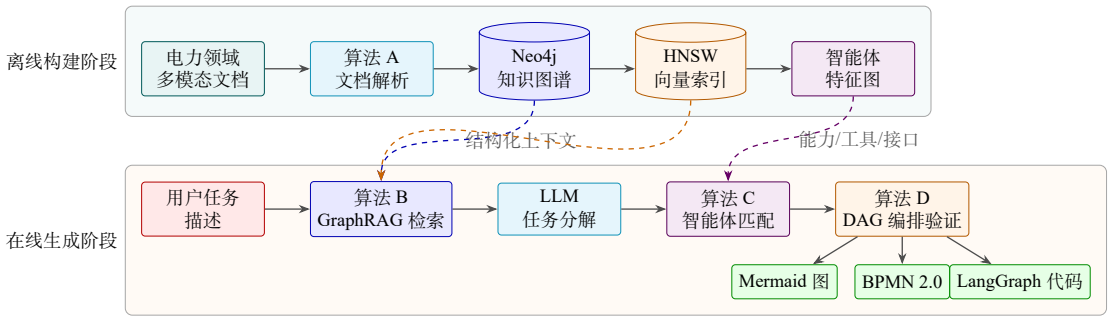

图 4-3 多模态 GraphRAG 驱动的智能体工作流生成系统架构

离线构建阶段：通过多模态文档解析流水线（算法 A）处理电力领域文档，构建多模态知识图谱并写入 Neo4j，建立向量索引，同时初始化智能体特征图。

该阶段一次构建，多次复用。

在线生成阶段：接收用户任务描述后，经 GraphRAG 检索（算法 B）获取领域上下文，随后通过子任务-智能体匹配（算法 C）完成任务分解和角色分配，最终由 DAG 编排与验证（算法 D）生成可执行工作流并导出。整个在线流程无需人工干预，30 任务全量评测中平均端到端延迟约为 9 至 22 秒（依基线配置、任务复杂度和 LLM 响应速度而定，详见第五章实验）。

4.5 本章小结

本章详细阐述了子任务-智能体混合匹配算法（算法 C）和工作流 DAG 编排与验证机制（算法 D）。算法 C 通过能力嵌入相似度、工具覆盖率和输入输出兼容性三维加权评分，系统化地将分解后的子任务映射到最优专用智能体，从根本上解决了角色分配随机性问题；算法 D 基于 LLM 推断数据流依赖，构建工作流 DAG，并通过 DAG 无环性、接口兼容性和任务完整性三层验证保证工作流质量，支持 Mermaid、BPMN 2.0 和 LangGraph 三种格式导出。结合第三章的算法 A 和 B，四项核心算法共同构成完整的智能体工作流自主生成方法体系。第五章将对所提方法在电力系统真实场景中的有效性进行全面实验评估。

5 实验与结果分析

本章在电力系统三类典型场景上对所提方法进行全面实验评估。首先介绍实验环境、数据集与基准任务，随后定义评估指标并说明步骤完整性指标的语义化升级，然后将本研究方法（ours_full）与三种基线方法在 30 条扩展基准任务上进行对比，最后通过消融实验和案例分析，定量定位每个算法模块的贡献与失效模式。

5.1 实验环境与数据集

5.1.1 实验环境

实验在以下软硬件环境中开展：操作系统为 Windows 11，处理器为 Intel Core i7，内存 16 GB。软件环境：Python 3.11，Neo4j Desktop 5.x（图数据库，含向量索引），sentence-transformers 2.x（BGE-M3 本地推理），DeepSeek Chat API（LLM 推理，兼容 OpenAI 接口）。BGE-M3 嵌入维度 1024，Neo4j 向量索引采用 HNSW 算法（$m = 16$，$ef_construction = 64$，余弦相似度）。

5.1.2 数据集

知识图谱与领域语料来源。离线知识图谱构建以 PowerGridQA 中的 NERC 运行经验、Power System Theory 与 Reasoning 三个子集为主要文本来源[33]，并参考 Electricity Knowledge Graph 的电力消费知识组织方式[34] 和 CIM-Graph 对 CIM 模型的图结构表达[35] 补充实体类型、设备关系和数据字段命名。通过算法 A 解析后写入 Neo4j，最终图谱包含 4,337 个节点和 5,228 条关系边，覆盖变压器、断路器、继电保护装置、负荷指标、可再生能源出力等核心对象及其故障类型和操作规程。

PowerGridQA 数据集。PowerGridQA 用于电力知识问答（KQA）和电网故障排查（OPS）场景。本文使用的本地版本包含 58,713 条问答记录，其中电力系统理论子集 53,138 条、NERC 运行经验子集 4,787 条、推理场景子集 788 条。KQA 任务主要来自理论与推理子集，OPS 任务主要根据 NERC 运行经验子集改

写为故障排查工作流需求。

OPSD 时序数据集。数据分析（DA）场景采用 Open Power System Data（OPSD）平台的时间序列数据[36]。本文本地实验文件为 `time_series_60min_singleindex.csv`，包含 50,401 条小时级记录和 300 个负荷、风电、光伏、电价等字段；实验脚本读取其中的统计摘要作为图谱文本节点，并构造短期负荷预测、峰谷差识别、新能源出力分析等数据分析工作流任务。

扩展数据来源。除 PowerGridQA 与 OPSD 外，数据准备阶段还核查了 ENTSO-E Transparency Platform[37]、Texas A&M 电网测试案例[38-39] 以及 OpenEI/OEDI 美国电力异常报告数据[40]。这些数据源分别用于补充欧洲实时运行数据、合成电网拓扑/潮流测试案例和美国停电事件记录，主要作为领域背景与基准任务设计的参考。

基准测试集。为避免仅依赖少量示例造成结论不稳，本文构造了包含 30 条任务的分层基准测试集，OPS、KQA、DA 三类场景各 10 条。每条样本均包含任务编号、数据来源、难度标签和金标准工作流（gold workflow），金标准工作流由 3 至 6 个步骤组成，并标注每步应分配的智能体。OPS 任务覆盖母线故障、线路跳闸、主变保护动作、单相接地、断路器失灵、N-1 越限、极端天气停电、SCADA 遥测异常等场景；KQA 任务覆盖继电保护、电力系统稳定性、SCADA 操作规程、标幺制、N-1 准则、CIM 模型等知识问答；DA 任务覆盖负荷预测、异常检测、新能源出力、电价相关性、备用裕度、停电事件统计和运行日报生成。表 5-1 给出了基准测试集的规模。

表 5-1　扩展基准测试集规模

场景	任务数	平均金标步骤数	主要数据来源
OPS	10	6.0	PowerGridQA-NERC、OpenEI/OEDI、Texas A&M、CIM-Graph
KQA	10	4.3	PowerGridQA、Electricity Knowledge Graph、CIM-Graph
DA	10	4.5	OPSD、ENTSO-E、OpenEI/OEDI
合计	30	4.9	多源电力公开数据集

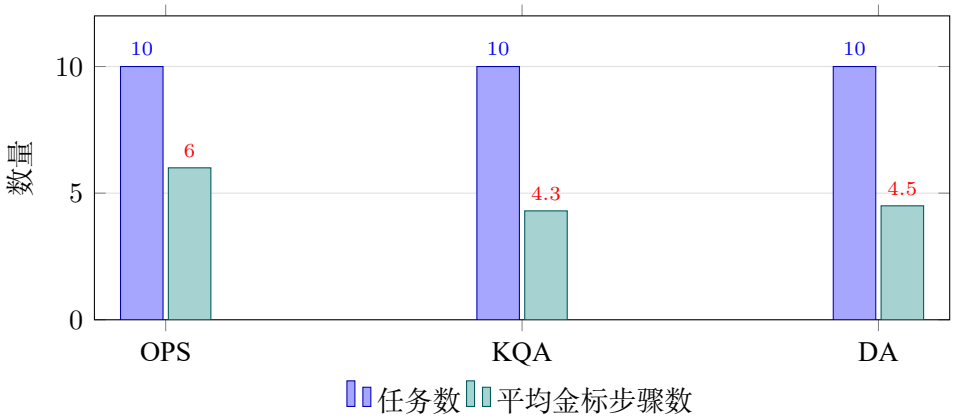

图 5-1　扩展基准测试集的任务规模与金标步骤规模

5.1.3　对比基线

本实验与三种基线方法进行对比：

- **pure_llm**：直接调用 LLM 生成工作流，无任何外部知识注入，步骤中的智能体字段由 LLM 自由填充。
- **vector_rag**：使用纯语义向量检索（max_hops $= 0$，即不进行子图扩展）获取上下文，其余流程与 ours_full 相同，含算法 C 匹配。
- **graphrag_nomatch**：使用完整 GraphRAG 检索（含 BFS 子图扩展），但子任务-智能体匹配采用随机分配（随机种子 42），不运行算法 C。

本研究完整方法（ours_full）采用算法 A 构建的知识图谱、算法 B 进行 GraphRAG 检索（$k = 5$，$h = 2$）、算法 C 进行三维评分匹配（$\alpha = 0.5$，$\beta = 0.3$，$\gamma = 0.2$）、算法 D 进行 DAG 编排与三层验证。这四种基线两两组合形成消融实验的设计矩阵：vector_rag 与 ours_full 的差异隔离了子图扩展（*算法 B 的 BFS 阶段*）的贡献，graphrag_nomatch 与 ours_full 的差异隔离了*算法 C* 的贡献。

实验复现与结果产物。端到端评估由 `scripts/run_eval_ops_workflow.py` 统一执行，支持 `--gold-file` 切换金标基准、`--scenes` 和 `--limit` 控制子集、`--baselines` 选择对比方法、`--skip-logic` 跳过 LLM 裁判以降低成本。脚本除逐任务 CSV 外，还自动生成分场景汇总表、全局均值表、95% 置信区间和 PGFPlots 数据文件；本章中的图 5-1、图 5-2、图 5-3、图 5-4 和图 5-5 由同一数据源自动刷新。

5.2 评估指标

定义以下四项量化评估指标：

（1）工作流可执行率（Executability，Exec）。判断生成工作流是否满足 DAG 结构有效性（无环性）以及每个步骤均有有效智能体分配。取值为 0 或 1。

（2）步骤完整性（Step Completeness，SC）。衡量生成工作流中步骤与金标准步骤的语义覆盖度。本文同时采用两种实现：

- **SC（语义版，主指标）**：将每个步骤的 action 文本经 BGE-M3 嵌入为 1024 维语义向量，计算金标步骤与生成步骤之间的两两余弦相似度，若任一生成步骤与某金标步骤的相似度超过阈值 $\tau_{sc} = 0.65$ 即视为覆盖，最终统计为召回率：

$$\mathrm{SC} = \frac{|\text{matched gold steps}|}{|\text{gold steps}|} \tag{5-1}$$

- **$\mathrm{SC}_{\mathrm{lex}}$（词汇版，对照指标）**：英文词元 + 中文字 n-gram 的 Jaccard 相似度，阈值 0.18。该指标作为初版实现保留以观测语义化改造的增益。

（3）角色分配准确率（Role Assignment Accuracy，RAA）。按照工作流拓扑顺序对齐生成步骤与金标准步骤，并将常见中文智能体名称、英文类名归一化为标准智能体标识后，判断生成步骤分配的智能体是否与金标准一致：

$$\mathrm{RAA} = \frac{|\text{steps with correct agent}|}{|\text{gold steps}|} \tag{5-2}$$

（4）逻辑正确性（Logical Correctness，Logic）。由 LLM 作为裁判，对生成工作流的逻辑顺序合理性、步骤操作可行性和整体流程完整性进行综合评分，分值范围 $[0, 1]$。同一工作流评分 3 次取均值以降低方差。

5.3 实验结果与分析

5.3.1 整体对比实验

表 5-2 展示了四种方法在 30 条基准任务上的全场景平均结果，分场景结果见表 5-3。所有方法在 Exec 上均达到 1.000，说明算法 D 的三层结构验证机制和基线方法的 JSON Schema 约束均能保证生成工作流为合法 DAG。下文聚焦于

RAA、SC 和 Logic 三项区分度更高的指标。

表 5-2　各方法在 30 条基准任务上的全场景平均结果

方法	Exec	SC	SC_{lex}	RAA	Logic
ours_full	1.000	0.768	0.196	0.296	0.711
pure_llm	1.000	0.696	0.091	0.000	**0.947**
vector_rag	1.000	**0.784**	0.181	**0.425**	0.912
graphrag_nomatch	1.000	0.784	0.180	0.056	0.868

表 5-3　各方法在三类场景上的角色分配准确率（RAA）

场景	ours_full	pure_llm	vector_rag	graphrag_nomatch
OPS	0.200	0.000	**0.250**	0.117
KQA	0.450	0.000	**0.615**	0.025
DA	0.237	0.000	**0.410**	0.025
均值	0.296	0.000	**0.425**	0.056

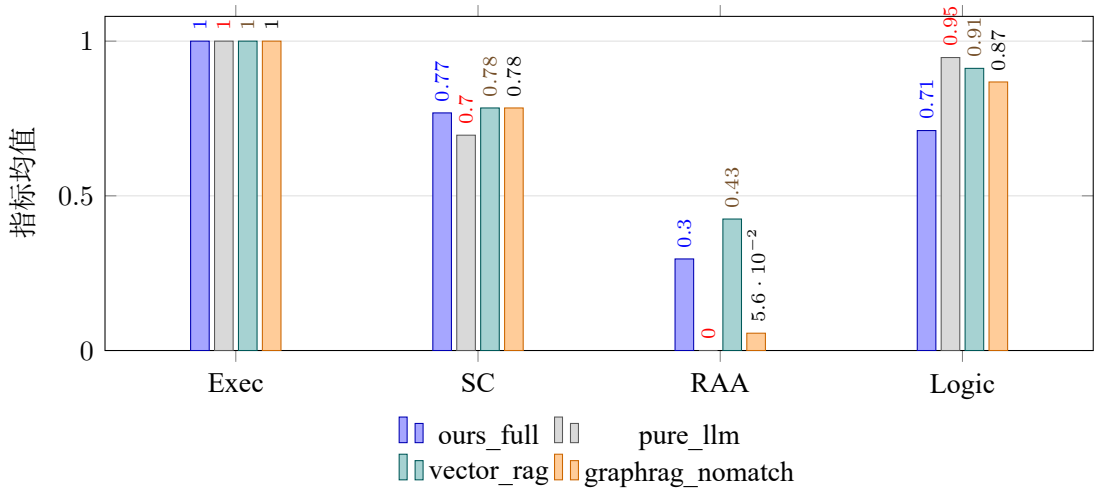

图 5-2　不同方法在四项工作流质量指标上的总体对比

角色分配准确率（核心指标）。vector_rag 在全场景均值上以 0.425 取得最佳成绩，其次是 ours_full（0.296），graphrag_nomatch（0.056）与 pure_llm（0.000）显著落后。这一结果可拆解为两条独立信号：

(1) **算法 C 的贡献是决定性的（+36.9 pp）**。vector_rag 与 graphrag_nomatch 的唯一差别是子任务-智能体匹配阶段——前者使用算法 C 的三维评分，后者使用随机分配。两者 RAA 之差 0.425 vs 0.056 直接量化了算法 C 的贡献，相当于将随机水平的角色分配提升至接近一半的命中率。在三个场景上这一规律均稳定成立（OPS：+13.3 pp，KQA：+59.0 pp，DA：+38.5 pp）。

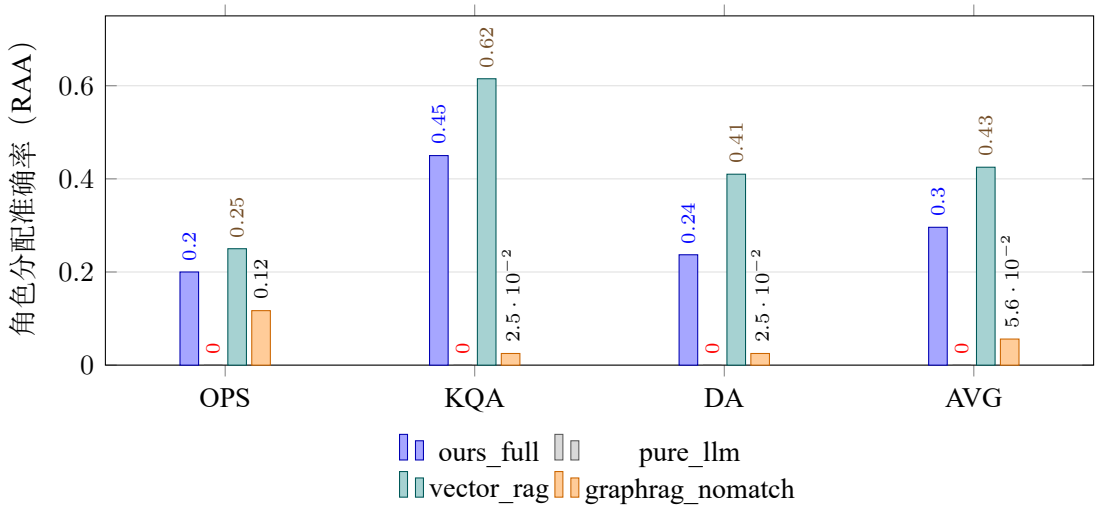

图 5-3　不同方法在各场景下的角色分配准确率对比

(2) **BFS 子图扩展（$h = 2$）在当前图谱上反而损害下游匹配（−12.9 pp）**。ours_full 与 vector_rag 的唯一差别是检索阶段是否进行 BFS 子图扩展，前者 RAA 反而比后者低 0.129。该现象在三个场景中一致出现（OPS：−5.0 pp，KQA：−16.5 pp，DA：−17.3 pp），并非单一场景的偶然波动。后续 5.3.3 节通过案例分析定位了该现象的成因。

步骤完整性。采用 BGE-M3 语义版 SC 后，所有方法的步骤覆盖度均落入 0.70 至 0.78 区间，与原词汇版 SC（均值 0.18 上下）相比有数倍提升。这一对比直接证明：原词汇版 SC 因无法识别”校验保护整定值”与”检查继电保护定值”这类专业术语近义，对中文电力工作流的覆盖度系统性低估；而语义版 SC 在同一批生成结果上揭示出方法间真实的覆盖差异。其中 vector_rag 与 graphrag_nomatch 的 SC 并列最高（0.784），说明知识图谱上下文确实有助于生成与金标更接近的步骤描述；ours_full 略低（0.768），与 BFS 扩展引入冗余概念、稀释了与金标重叠的步骤一致；pure_llm 最低（0.696），符合无外部知识时的预期。

逻辑正确性的裁判偏差。pure_llm 取得了最高的 Logic 分（0.947），高于含图谱的方法约 5–24 个百分点，这一现象与 RAA 排序完全相反。结合人工抽查发现，LLM-as-Judge 对简洁、表述通顺的工作流给出更高分数，而对带有专业术语和领域操作的步骤评分相对保守；同时裁判主要观察”步骤间顺序是否合理”而非”角色分配是否专业”，因此该指标在本任务下存在系统性偏差。本研究在结论中将 RAA 与 SC 列为更可靠的客观指标，并将 Logic 视为参考性辅助指标，

未来工作中应引入人工评分或多模型集成裁判替换之。

运行效率。图 5-4 展示了不同方法的平均端到端延迟。pure_llm 无需检索和智能体匹配，平均延迟最低（9.34 s）；vector_rag 增加向量检索和算法 C 匹配，延迟上升至 14.34 s；graphrag_nomatch 和 ours_full 均包含 BFS 子图扩展，延迟分别为 19.08 s 和 21.78 s。也就是说，相比于 vector_rag，BFS 子图扩展平均增加 7.4 s 延迟，却造成 12.9 pp 的 RAA 下降——在当前图谱规模下，子图扩展是时间和质量双重负贡献。

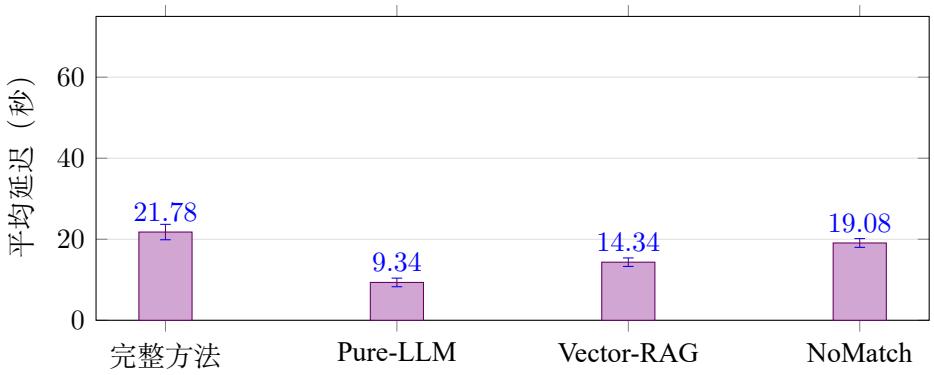

图 5-4　不同方法的端到端平均生成延迟

5.3.2　消融实验

为分别量化算法 C 与 BFS 子图扩展的独立贡献，按照以下消融维度对 RAA 全场景均值进行分析（表 5-4）：

表 5-4　消融实验：角色分配准确率（全场景均值，n=30）

消融配置	算法 C	BFS 子图扩展	RAA 均值
NoSubgraph（vector_rag）	✓	×	**0.425**
ours_full	✓	✓	0.296
NoMatch（graphrag_nomatch）	×	✓	0.056
NoKG（pure_llm）	×	×	0.000

算法 C 是 RAA 的决定性因素。在保留 BFS 扩展的条件下，引入算法 C 使 RAA 从 0.056（NoMatch）提升至 0.296（ours_full），增幅 24.0 pp；在不使用 BFS 扩展的条件下，引入算法 C 使 RAA 从 0.000（NoKG，pure_llm 在归一化后所有自由命名的智能体均无法匹配标准 ID）提升至 0.425（NoSubgraph），增幅 42.5 pp。两种条件下算法 C 均贡献了 RAA 的主要部分，验证了三维加权评分函数

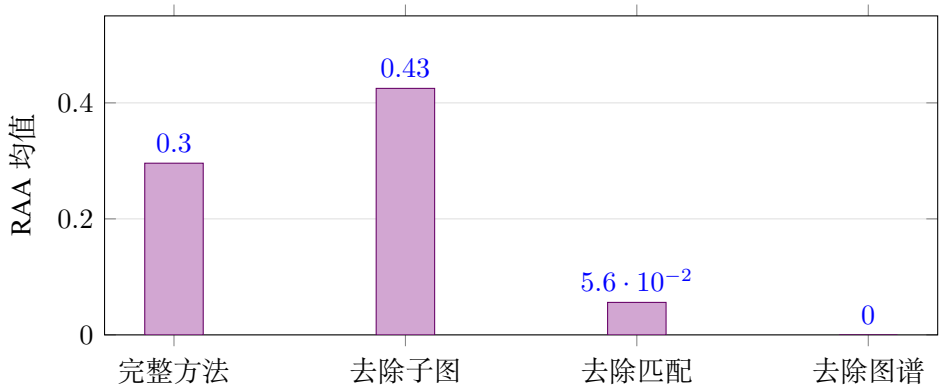

图 5-5　消融实验中不同模块配置的角色分配准确率

（能力嵌入相似度 + 工具覆盖率 + 输入输出兼容性）相对于 LLM 隐式分配和随机分配的根本优势。

BFS 子图扩展未带来稳定增益。在保留算法 C 的条件下，引入 BFS 扩展反而使 RAA 从 0.425（NoSubgraph）下降至 0.296（ours_full），降幅 12.9 pp；在不使用算法 C 的条件下，BFS 扩展使 RAA 从 0.000（NoKG）提升至 0.056（NoMatch），增幅微小且数值仍接近随机水平。综合而言，BFS 扩展未能在本基准下提供稳定的增量收益，反而在与算法 C 组合时引入噪声节点稀释下游匹配信号。

算法 C 与 BFS 扩展存在负向交互。NoSubgraph 相对于 ours_full 的 RAA 提升与 NoMatch 相对于 NoKG 的 RAA 提升不能简单相加：BFS 扩展把检索召回的节点数从 5（top-k）扩展到平均 97.5（30 任务 × 2 个含 BFS 基线共 60 次采样，最大 169，最小 47），即每次检索引入约 *90* 个新节点，远高于按平均度数 2.4 估计的理论值——这是由于部分电力概念节点（如保护动作、故障类型）作为”枢纽”节点连通度显著高于均值。这些扩展节点中，大量与任务弱相关的”恢复”、”备用”、”统计”类概念被一并卷入上下文。LLM 在阅读扩展后的上下文进行任务分解时，会受这些噪声概念引导生成超出最优步骤数的子任务，使得算法 C 不得不为这些”额外”子任务匹配次优智能体，从而稀释整体准确率。该解释在 5.3.3 节的两个 OPS 案例中得到了直接印证。

权重敏感性分析。为评估算法 C 三维权重 (α, β, γ) 的选择是否合理，本文额外开展了权重网格扫描实验。具体做法是在 15 条任务子集上预先固化检索结果与子任务分解结果（与权重无关），随后对 $\alpha + \beta + \gamma = 1$ 离散步长 0.1 的全部 66 个权重组合分别计算 RAA。脚本 `scripts/run_weight_sweep.py` 在缓存模式下

完整扫描耗时约 1 分钟，结果如图 5-6 所示。论文采用的默认权重 $(0.5, 0.3, 0.2)$ 在该子集上取得 RAA $= 0.303$，最优配置位于 $\alpha = 0.1$、$\beta \in [0.4, 0.8]$ 区域，对应的 RAA $= 0.348$，相对默认配置高出 4.5 个百分点；而 $\alpha = 0$（完全去除能力嵌入相似度）的全部组合均落在 0.251 及以下，明显劣于其他配置。该敏感性结果传递两条信息：（1）α 不能为零——能力嵌入相似度是不可缺省的维度；（2）当前实现下 α 的最佳取值偏低，原因可能是电力领域专用智能体的能力描述高度同质，BGE-M3 在该子领域的判别力相对有限，工具覆盖率维度反而提供了更强的辨别信号。鉴于本文的核心主张为" 算法 C 提供 +36.9 pp 的稳定提升"，且各非零 α 区间内的 RAA 跨度仅约 5–6 pp，所采用的默认权重处于该敏感区间内，结论仍然成立；进一步的权重精调留待后续工作。

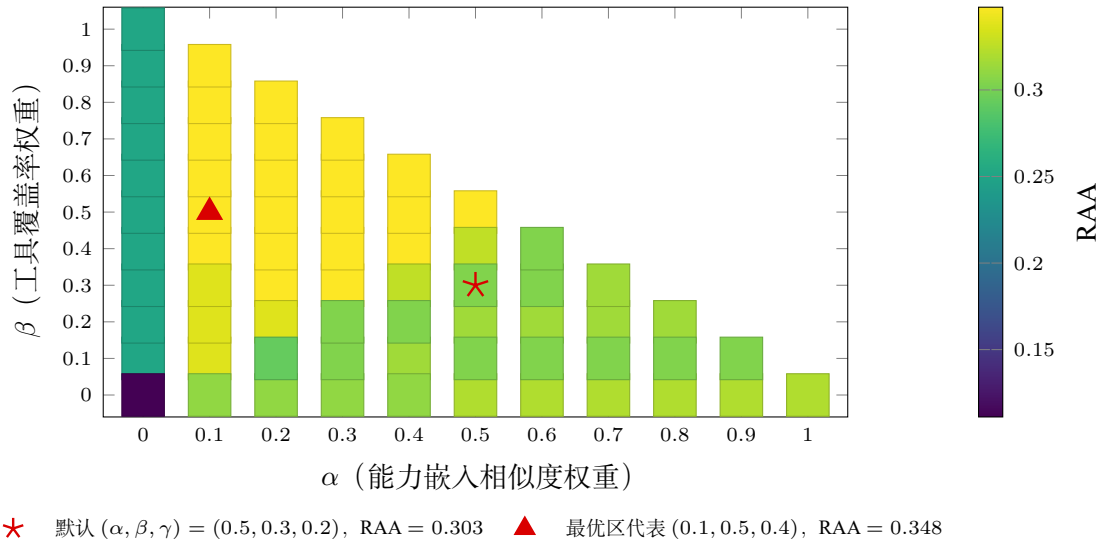

图 5-6　算法 C 三维权重 (α, β) 网格扫描的 RAA 热力图（$\gamma = 1 - \alpha - \beta$，15 任务子集）

5.3.3　案例分析

为定位 BFS 子图扩展导致 ours_full 角色准确率落后于 vector_rag 的具体失效模式，下面以两条 OPS 任务为例进行剖析。

案例一：220 kV 线路距离保护 I 段动作跳闸（ops_002）。金标工作流为 6 步：故障检测 → 拓扑分析 → 保护校验 → 潮流分析 → 恢复规划 → 报告生成。vector_rag 生成 6 个步骤，正确分配 fault_detector、protection_checker、

restoration_planner（两次）、report_generator，RAA = 0.667。ours_full 仅生成 5 个步骤，缺失 topology_analyzer 和 power_flow_analyzer 两个关键角色，并将原本应由 protection_checker 执行的”评估断路器及辅助设备状态”误分配给 restoration_planner，RAA = 0.333。检查 BFS 扩展上下文发现，子图引入了隔离故障、恢复供电、操作票等大量”恢复”概念节点，使 LLM 在任务分解时过度强调恢复阶段，跳过了拓扑与潮流校验这两个非”恢复”性步骤。

案例二：CIM 模型导入后开关状态校核（ops_010）。金标工作流为 6 步：拓扑分析 → 异常检测 → 故障判别 → 潮流校核 → 修正发布 → 报告生成。vector_rag 正确将首步分配给 topology_analyzer 并匹配出 anomaly_detector，RAA = 0.333。ours_full 将首步误分配给 anomaly_detector，并将后续步骤分配给 regulation_checker，缺少 topology_analyzer 作为唯一首步的判定，RAA = 0.000。子图扩展将 SCADA 异常、停电事件等”异常检测”相关节点排在重排靠前位置，使 LLM 在任务分解时即把”获取数据”环节定位为”数据质量检查”，进而触发 anomaly_detector 而非 topology_analyzer。

两个案例共同揭示了 BFS 子图扩展的失效模式：当查询本身已有较强的语义聚焦时，扩展引入的次要相关节点会改变下游 LLM 对任务结构的理解，进而扰动子任务粒度与角色分配。这一观察与 5.3.2 节的消融数据相互印证，提示后续可探索基于查询难度的自适应跳数控制（如 $h \in \{0, 1, 2\}$ 由查询置信度动态选择）。

5.3.4 系统运行实证

为佐证前述实验数据均来自系统的真实端到端运行，下面给出三方面的运行证据：评测脚本的终端日志、生成工作流的 DAG 结构以及 Neo4j 中实际命中的子图。

终端运行日志。图 5-7 截取了 30 任务全量评测脚本在 ops_002 上的实际输出。每条任务日志包含 GraphRAG 检索阶段的中间统计（种子节点数、BFS 扩展规模、重排后保留的节点数）和四种基线的逐项指标。从中可见 ours_full 与 graphrag_nomatch 的检索阶段返回相同规模的子图（95 节点 / 139 边 / 重排后 18 节点），而 vector_rag 在 `max_hops=0` 时仅保留 5 个种子节点，验证了 5.3.2 节关

于 BFS 扩展规模的描述。

```
$ python scripts/run_eval_ops_workflow.py
================================================================
[OPS] 220kV 线路距离保护 I 段动作跳闸，重合闸失败，...
================================================================
[retriever] 种子节点：5 个
[retriever] 扩展后子图：95 节点，139 边
[retriever] 重排序后：18 节点
 [ours_full]        exec=1.00 sc=0.67 role=0.33 logic=0.60 steps=5  22.3s
 [pure_llm]         exec=1.00 sc=0.50 role=0.00 logic=0.90 steps=5   8.6s
[retriever] 种子节点：5 个
[retriever] 扩展后子图：5 节点，0 边（vector_rag，max_hops=0）
[retriever] 重排序后：3 节点
 [vector_rag]       exec=1.00 sc=0.67 role=0.67 logic=0.90 steps=6  18.1s
[retriever] 种子节点：5 个
[retriever] 扩展后子图：95 节点，139 边
[retriever] 重排序后：18 节点
 [graphrag_nomatch]exec=1.00 sc=0.67 role=0.00 logic=0.90 steps=5  17.1s
```

图 5-7　30 任务全量评测脚本在 OPS-002 任务上的终端运行日志（节选）

生成工作流的 DAG 结构。图 5-8 展示了 ours_full 在 ops_002 上实际生成、并经算法 D 三层验证-修复后导出的工作流 DAG。该 DAG 包含 5 个步骤、2 条由 LLM 隐式数据流推断得到的依赖边，对应金标 6 步流程中 fault_detector 与 protection_checker 两步被正确分配，restoration_planner 的过度激活则印证了案例一中 BFS 扩展引入”恢复”语义偏向的失效模式。该图同时也是算法 D 端到端导出的 Mermaid 工作流形态。

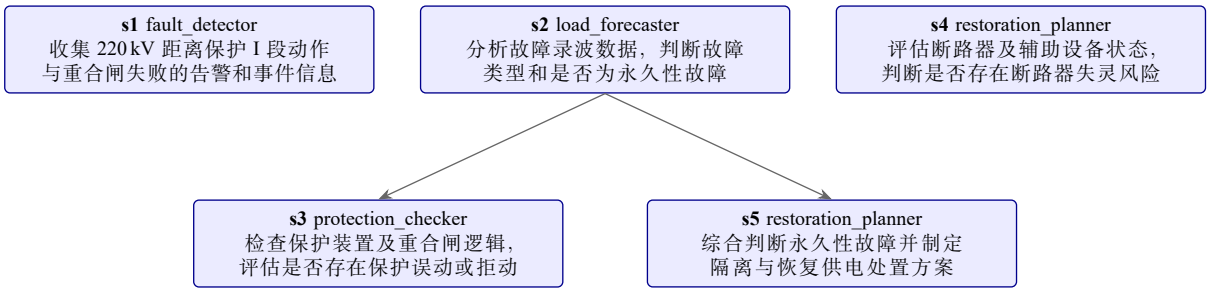

图 5-8　ours_full 在 OPS-002 任务上生成的工作流 DAG

Neo4j 中的实际子图。图 5-9 给出了在算法 B 检索过程中实际命中的子图样例。种子节点是 PowerGridQA-NERC 中关于 LL20231101 电流差动保护逻辑的文本节点，1 跳邻域包含 6 个实体节点（电流差动保护、NERC 报告标识、故障、跳闸、间歇性退化信号、230 kV 输电线），2 跳邻域进一步引入若干主题相关的文本节点。该结构与 5.3.2 节给出的”$h = 2$ 平均扩展 97.5 节点、最大 169” 的统

计吻合，也直观显示了为何子图扩展容易把”跳闸/恢复”等 1 跳实体连同其文档汇入上下文，进而稀释下游 LLM 的任务分解焦点。

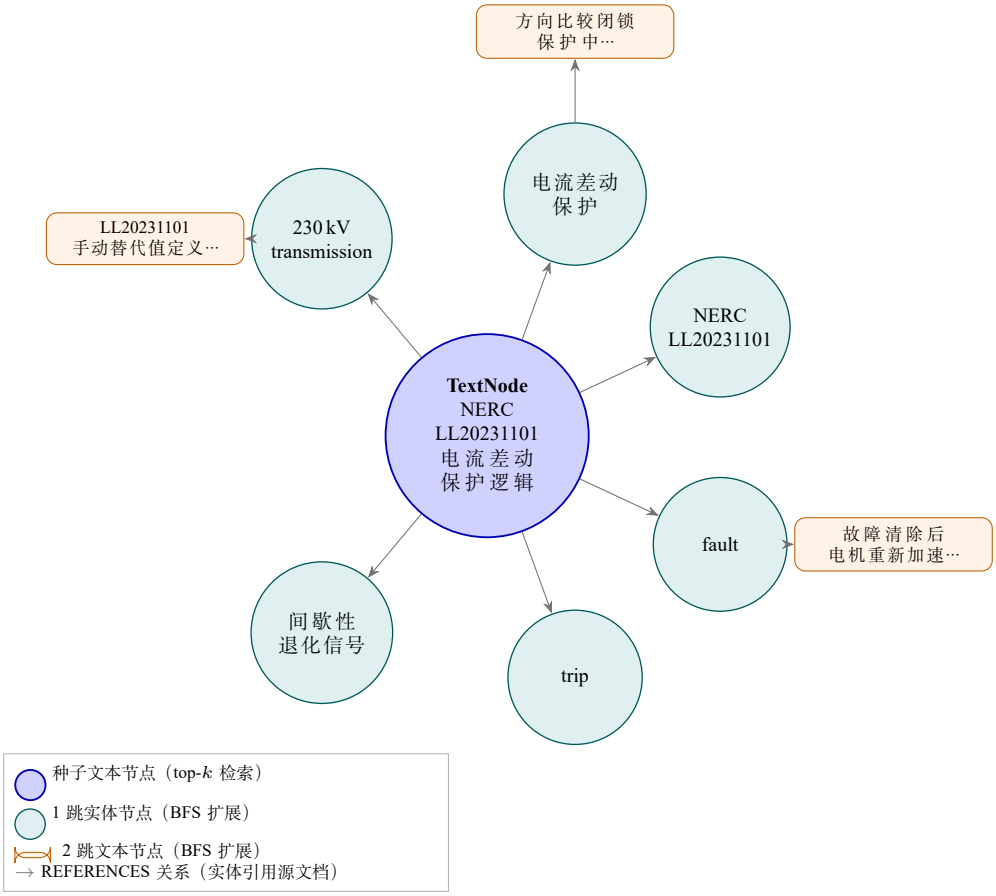

图 5-9　算法 B 在 Neo4j 中检索得到的实际子图（以电流差动保护为种子，2 跳扩展）

5.4　本章小结

本章在 30 条电力领域基准任务上系统评估了所提方法与三种基线。主要结论如下：（1）所有方法在 Exec 上均达到 1.000，验证了算法 D 三层验证机制能稳定输出合法 DAG；（2）算法 C 的三维量化匹配是 RAA 的决定性因素，相比未匹配基线（graphrag_nomatch）将全场景 RAA 从 0.056 提升至 0.425（+36.9 pp），并在 OPS、KQA、DA 三类场景上均稳定成立；（3）BFS 子图扩展（$h = 2$）在当前 4,337 节点的电力图谱上未带来稳定增益，反而由于引入噪声节点改变了 LLM 的任务分解粒度，使 ours_full 较 vector_rag 的 RAA 下降 12.9 pp；（4）步骤完整性指标从词汇版升级为 BGE-M3 语义版后，覆盖率从 0.18 量级提升至 0.77

量级，更准确地反映了中文电力领域工作流的步骤覆盖情况；（5）LLM-as-Judge 在 Logic 指标上存在系统性偏差，简洁工作流被高估，建议未来以人工评分或多模型集成裁判替代。综合而言，实验数据支持以”基于图谱的语义检索 + 三维量化匹配”作为本研究的主要贡献，而 BFS 子图扩展的有效场景需配合自适应跳数策略与图谱规模扩展进一步研究。

6 结论与展望

6.1 研究结论

本文围绕”多模态图检索增强的智能体工作流自主生成”这一核心命题，构建了从多模态知识图谱构建到可执行工作流输出的完整方法体系，并通过 30 条电力领域基准任务的全量实验对各模块进行了独立量化分析。研究结论总结如下：

（1）**多模态知识图谱构建方法（算法 A）**。提出面向电力领域的多模态文档解析流水线，支持文本、图像和流程图三种模态的统一实体-关系抽取，利用 BGE-M3 嵌入模型（1024 维）生成语义向量，基于 Neo4j 图数据库建立 HNSW 向量索引，最终构建包含 4,337 个节点和 5,228 条关系的电力知识图谱。该图谱为后续检索与匹配提供了一致的领域知识底座。当前实验中图像与流程图链路尚未在端到端任务上独立验证，相关多模态收益的量化将在未来工作中开展。

（2）**基于子图扩展的 GraphRAG 检索方法（算法 B）**。提出”语义向量检索种子节点 + BFS 子图扩展 + LLM 重排序”的两阶段检索策略，并设计结构化的图-文上下文格式用于 LLM 推理。实验结果显示：在当前 4,337 节点的图谱规模下，向量种子检索（vector_rag）已能为下游匹配提供足够上下文，BFS 子图扩展（$h = 2$）反而平均下降 12.9 个百分点的角色分配准确率，并伴随约 7.4 s 的额外延迟。案例分析表明，扩展引入的次要相关节点会改变 LLM 对任务结构的理解，从而扰动子任务粒度。该结果提示子图扩展的有效场景需要配合自适应跳数控制和更大规模图谱，相关方法本研究保留为未来研究方向。

（3）**子任务-智能体混合匹配算法（算法 C）**。提出三维加权评分函数，从能力嵌入相似度（$\alpha = 0.5$）、工具覆盖率（$\beta = 0.3$）和输入输出兼容性（$\gamma = 0.2$）三个维度系统化匹配子任务与智能体，并设计软约束回退机制保证全覆盖。实验表明，算法 C 是 RAA 的决定性因素：在保留 BFS 扩展的条件下，算法 C 将 RAA 从 0.056（随机分配基线）提升至 0.296（+24.0 pp）；在不使用 BFS 扩展的条件下，提升幅度进一步扩大至 +42.5 pp，使 RAA 达到 0.425。该模块在 OPS、KQA、DA 三类场景下均稳定贡献准确率提升，是本研究的主要技术贡献。

（4）**工作流 DAG 编排与验证机制（算法 D）**。设计基于 LLM 的隐式数据流依赖推断方法构建工作流 DAG，并实现 DAG 无环性、接口兼容性和任务完整性三层迭代验证-修复机制（最大迭代次数 5）。实验中所有方法的工作流可执行率均达到 100%，验证了 DAG 验证-修复闭环的有效性。

综上，本研究证明了”基于图谱的语义检索 + 三维量化智能体匹配”是提升专业领域工作流自动生成质量的有效路径，特别是算法 C 在多个场景下表现出稳定且可解释的角色分配能力，为电力调度、故障应急处置、知识服务等场景提供了可行的工程参考。

6.2 研究局限性

本研究在实验环节坚持以全量数据如实呈现各模块的真实效果，揭示出以下局限性：

（1）**BFS 子图扩展未带来稳定增益**。当前 4,337 节点的图谱规模相对有限，BFS $h = 2$ 扩展引入的节点中相当一部分与查询主题弱相关，反而稀释了下游匹配信号。这一观察提示子图扩展并非”越多越好”，需要结合查询置信度自适应控制跳数，并在更大规模、更稀疏的图谱上重新评估其增益边界。

（2）**LLM-as-Judge 存在系统性偏差**。Logic 指标采用 LLM 裁判时，简洁、表述通顺的工作流被一致性地高估（pure_llm 取得 0.947 的 Logic 分但 RAA 为 0.000），与 RAA 排序完全相反。本研究在结论中将 RAA 与 SC 视为更可靠的客观指标。Logic 指标的修正应通过引入人工评分、多模型集成裁判或细化裁判提示词等方式实现。

（3）**多模态链路未在端到端任务上独立量化**。算法 A 的图像与流程图处理路径已设计完毕，但当前实验主要使用文本来源的 PowerGridQA 与 OPSD 构建图谱节点。多模态信息在工作流生成任务上的独立增益（例如增加单线图后 OPS 任务 RAA 是否提升）尚未单独验证，是评估的一个空白。

（4）**评估规模与领域覆盖**。本文已将初始 6 条人工标注任务扩展为 30 条分层基准任务，但相较真实电网运行中的故障类型、知识问题和数据分析需求，样本规模仍然有限。未来研究应继续扩展至数百条任务，并引入自动化预标注与人工复核流程降低标注成本。

（5）实时性与资源消耗。算法 C 的 IO 兼容性维度当前以静态字符串匹配实现，相较论文方法形式中的 LLM 判别版本牺牲了一些精度但大幅降低了 LLM 调用频次；算法 D 的依赖推断和验证修复仍需 LLM 调用，端到端延迟约为 9 至 22 秒。对于实时性要求高的场景（如电网事故紧急处置），需进一步通过缓存与本地小模型替换降低延迟。

6.3 未来研究方向

基于上述局限性，未来研究可从以下方向展开：

（1）自适应子图扩展策略。本研究观测到固定 $h = 2$ 的 BFS 扩展未能稳定提升下游质量。未来可设计基于查询置信度的自适应跳数选择器（$h \in \{0, 1, 2, 3\}$）、基于关系类型的方向性遍历，或将 GraphRAG 中的社区摘要机制与子图扩展结合，让子图规模随任务复杂度动态调整。

（2）多模态链路独立验证。补充包含电力单线图、保护配置图和操作流程图的图像数据集，量化多模态节点对任务分解和角色匹配的独立贡献，验证算法 A 中视觉语言模型 + 实体抽取 + 跨模态对齐流水线的端到端价值。

（3）IO 兼容性的语义化重构。将算法 C 中的字符串包含式 IO 兼容性升级为基于嵌入的语义判别或 LLM 三级评分，并通过缓存机制控制 LLM 调用次数，在保持精度的同时降低延迟。

（4）工作流执行反馈闭环。引入强化学习或经验回放机制，将工作流实际执行结果（步骤成功/失败、输出质量）反馈给匹配算法，实现 α, β, γ 权重参数的自适应优化。

（5）跨领域通用性验证。将所提方法迁移至医疗诊断、法律文书分析和金融风控等其他专业领域，验证三维量化匹配在不同图谱结构下的稳定性，并研究领域适配所需的最小知识图谱规模。

（6）端到端工作流执行验证。当前实验仅评估工作流生成质量，未对 LangGraph 导出的工作流进行实际执行验证。未来可搭建电力系统仿真环境，接入真实智能体并执行生成工作流，以任务最终完成率为终极评估指标，形成更闭环的评估体系。

参考文献

[1] OpenAI. GPT-4 Technical Report[J]. arXiv preprint arXiv:2303.08774, 2023.

[2] Anthropic. Claude 3 Model Card[Z]. https://assets.anthropic.com/m/61e7d27f8c8f5919/original/Claude-3-Model-Card.pdf. 2024.

[3] 罗文, 王厚峰. 大语言模型评测综述[J]. 中文信息学报, 2024, 38(1): 1-23.

[4] LangChain AI. LangGraph: Building Stateful Multi-Actor Applications with LLMs[Z]. https://github.com/langchain-ai/langgraph. 2024.

[5] WU Q, BANSAL G, ZHANG J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[C]//arXiv preprint arXiv:2308.08155. 2023.

[6] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.

[7] EDGE D, TRINH H, CHENG N, et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization[J]. arXiv preprint arXiv:2404.16130, 2024.

[8] NERC. NERC Lessons Learned: Protective Relay System Misoperations[Z]. https://www.nerc.com/pa/rrm/ea/Lessons%20Learned%20Document%20Library. 2023.

[9] 肖发龙, 吴岳忠, 沈雪豪, 等. 基于深度学习和知识图谱的变电站设备故障智能诊断[J/OL]. 电力建设, 2022, 43(3): 66-74. DOI: 10.12204/j.issn.1000-7229.2022.03.008.

[10] 姚一凡, 赵晋泉, 张沛超, 等. 基于知识图谱和溯因推理的智能电网故障诊断方法[J/OL]. 现代电力, 2025. DOI: 10.19725/j.cnki.1007-2322.2024.0314.

[11] ASAI A, WU Z, WANG Y, et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection[J]. arXiv preprint arXiv:2310.11511, 2023.

[12] YAN S Q, GU J C, ZHU Y, et al. Corrective Retrieval Augmented Generation[J]. arXiv preprint arXiv:2401.15884, 2024.

[13] JEONG S, BAEK J, CHO S, et al. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity[J]. arXiv preprint arXiv:2403.14403, 2024.

[14] 徐增林, 盛泳潘, 贺丽荣, 等. 知识图谱技术综述[J/OL]. 电子科技大学学报, 2016, 45(4): 589-606. DOI: 10.3969/j.issn.1001-0548.2016.04.012.

[15] 刘知远, 孙茂松, 林衍凯, 等. 知识表示学习研究进展[J/OL]. 计算机研究与发展, 2016, 53(2): 247-261. DOI: 10.7544/issn1000-1239.2016.20160020.

[16] SEN P, MAVADIA S, SAFFARI A. Knowledge Graph-Augmented Language Models for Complex Question Answering[C/OL]//Proceedings of the 1st Workshop on Natural Language Reasoning and Structured Explanations. 2023: 1-8. DOI: 10.18653/v1/2023.nlrse-1.1.

[17] SARTHI P, ABDULLAH S, TULI A, et al. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval[J]. arXiv preprint arXiv:2401.18059, 2024.

[18] HU Y, LEI Z, ZHANG Z, et al. GRAG: Graph Retrieval-Augmented Generation [J]. arXiv preprint arXiv:2405.16506, 2024.

[19] GUTIERREZ B J, SHU Y, GU Y, et al. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models[J]. arXiv preprint arXiv:2405.14831, 2024.

[20] 曹荣荣, 柳林, 于艳东, 等. 融合知识图谱的大语言模型研究综述[J/OL]. 计算机应用研究, 2025, 42(8): 2255-2266. DOI: 10.19734/j.issn.1001-3695.2024.12.0532.

[21] HONG S, ZHUGE M, CHEN J, et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework[J]. arXiv preprint arXiv:2308.00352, 2023.

[22] QIAO B, LI L, ZHANG X, et al. TaskWeaver: A Code-First Agent Framework [Z]. arXiv preprint arXiv:2311.17541. 2023.

[23] LI G, HAMMOUD H A A K, ITANI H, et al. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society[J]. Advances in Neural Information Processing Systems, 2023, 36: 51991-52008.

[24] CrewAI Inc. CrewAI: Framework for Orchestrating Role-Playing, Autonomous

AI Agents[Z]. https://github.com/crewAIInc/crewAI. 2024.

[25] CHENG Y, ZHAO H, ZHOU X, et al. A Large Language Model for Advanced Power Dispatch[J/OL]. Scientific Reports, 2025, 15(8925). DOI: 10.1038/s41598-025-91940-x.

[26] RADFORD A, KIM J W, HALLACY C, et al. Learning Transferable Visual Models From Natural Language Supervision[C]//Proceedings of the International Conference on Machine Learning. 2021: 8748-8763.

[27] LIU H, LI C, WU Q, et al. Visual Instruction Tuning[J]. Advances in Neural Information Processing Systems, 2023, 36.

[28] ZHAO Z, KANG H, WANG B, et al. DocLayout-YOLO: Enhancing Document Layout Analysis through Diverse Synthetic Data and Global-to-Local Adaptive Perception[Z]. arXiv preprint arXiv:2410.12628. 2024.

[29] SMOCK B, PESALA R, ABRAHAM R. Pubtables-1m: Towards comprehensive table extraction from unstructured documents[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 4634-4642.

[30] LIU F, WANG X, YAO W, et al. MMC: Advancing Multimodal Chart Understanding with Large-Scale Instruction Tuning[J]. arXiv preprint arXiv:2311.10774, 2023.

[31] CHEN J, XIAO S, ZHANG P, et al. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation[J]. arXiv preprint arXiv:2402.03216, 2024.

[32] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models[J]. Advances in Neural Information Processing Systems, 2022, 35: 24824-24837.

[33] HANNAAN A, KHANDAKAR A, HOSSEN M F, et al. A Structured Question–Answer Dataset for Large Language Models in Power System Dynamics and Operations[J/OL]. IEEE Access, 2026, 14: 8391-8407. DOI: 10.1109/ACCESS.2026.3652625.

[34] HANŽEL V, BERTALANIČ B, FORTUNA C. Towards Data-Driven Electricity

Management: Multi-Region Uniform Data and Knowledge Graph[J/OL]. Scientific Data, 2025, 12(1): 38. DOI: 10.1038/s41597-024-04310-z.

[35] ANDERSON A, ALLWARDT C, STEPHAN E. CIMantic Graphs[Z/OL]. [Computer Software] https://github.com/PNNL-CIM-Tools/CIM-Graph. 2024. DOI: 10.11578/dc.20240507.3.

[36] WIESE F, SCHLECHT I, BUNKE W D, et al. Open Power System Data: Frictionless Data for Electricity System Modelling[J/OL]. Applied Energy, 2019, 236: 401-409. DOI: 10.1016/j.apenergy.2018.11.097.

[37] HIRTH L, MÜHLENPFORDT J, BULKELEY M. The ENTSO-E Transparency Platform: A Review of Europe's Most Ambitious Electricity Data Platform [J/OL]. Applied Energy, 2018, 225: 1054-1067. DOI: 10.1016/j.apenergy.2018.04.048.

[38] Texas A&M University. Electric Grid Test Cases[Z]. https://electricgrids.engr.tamu.edu/electric-grid-test-cases/. Accessed: 2026-04-24. 2026.

[39] BIRCHFIELD A B, SCHWEITZER E, ATHARI M H, et al. A Metric-Based Validation Process to Assess the Realism of Synthetic Power Grids[J/OL]. Energies, 2017, 10(8): 1233. DOI: 10.3390/en10081233.

[40] Open Energy Data Initiative. Event-Correlated Outage Dataset in America[Z]. https://data.openei.org/submissions/6458. Accessed: 2026-04-24. 2026.

致 谢

至此，我的大学生涯走到了终点。初见夏至，满怀憧憬；再见盛夏，已是离别。

首先，衷心感谢指导教师莫益军教授，在论文研究的全过程中，莫老师给予了悉心指导和宝贵的学术启迪。从课题方向的确定、研究思路的梳理，到实验方案的设计和论文写作的完善，莫老师的严谨治学态度和深厚学识使我受益匪浅。

其次，感谢华中科技大学计算机科学与技术学院为我提供了优良的学习和研究环境。感谢实验室的同学们在日常讨论中给予的灵感和支持，让我在遇到困难时能够相互鼓励、共同进步。

感谢开源社区和相关项目的贡献者：Neo4j、LangGraph、sentence-transformers、DeepSeek 等工具和平台为本研究提供了重要的技术基础；PowerGridQA、OPSD、Electricity Knowledge Graph、CIM-Graph、ENTSO-E、Texas A&M 电网测试案例和 OpenEI/OEDI 等数据集或平台的发布者为实验设计与领域验证提供了数据支撑。

最后，感谢父母多年来对我求学之路的理解和支持。正是家人无私的关怀与鼓励，使我能够专心致力于学业，顺利完成本科阶段的学习。

本研究仍存在若干不足之处，有待后续工作进一步完善。谨以此文献给所有给予我帮助和支持的人。